

Verifiable E-Voting with a Trustless Bulletin Board

Daniel Rausch

*Institute of Information Security
University of Stuttgart
Stuttgart, Germany
daniel.rausch@sec.uni-stuttgart.de*

Nicolas Huber

*Institute of Information Security
University of Stuttgart
Stuttgart, Germany
nicolas.huber@sec.uni-stuttgart.de*

Ralf Küsters

*Institute of Information Security
University of Stuttgart
Stuttgart, Germany
ralf.kuesters@sec.uni-stuttgart.de*

Abstract—Voter privacy and end-to-end (E2E) verifiability are critical features of electronic voting (e-voting) systems to safeguard elections. To achieve these properties commonly a perfect bulletin board (BB) is assumed that provides consistent, reliable, and tamper-proof storage and transmission of voting data. However, in practice, BBs operate in asynchronous and unreliable networks, and hence, are susceptible to vulnerabilities such as equivocation attacks and dropped votes, which can compromise both verifiability and privacy. Although prior research has weakened the perfect BB assumption, it still depends on trusting certain BB components.

In this work, we present and initiate a formal exploration of designing e-voting systems based on fully untrusted BBs. For this purpose, we leverage the notion of accountability and in particular use accountable BBs. Accountability ensures that if a security breach occurs, then cryptographic evidence can identify malicious parties. Fully untrusted BBs running in asynchronous networks bring new challenges. Among others, we identify several types of attacks that a malicious but accountable BB might be able to perform and propose a new E2E verifiability notion for this setting. Based on this notion and as a proof of concept, we construct the first e-voting system that is provably E2E verifiable and provides vote privacy even when the underlying BB is fully malicious. This establishes an alternative to traditional e-voting architectures that rely on (threshold) trusted BB servers.

Index Terms—E-Voting, Accountability, Bulletin Board

I. INTRODUCTION

As democratic societies increasingly adopt electronic voting (e-voting) systems to enhance the accessibility, convenience, and efficiency of elections, the challenge of ensuring vote privacy and end-to-end (E2E) verifiability has gained significant research attention. Vote privacy is essential for maintaining the confidentiality of individual votes, while E2E verifiability allows everyone, including voters, candidates, and even external observers, to confirm that the election result reflects the intended votes of all voters. Both properties are central to protecting the integrity of elections, against accidental programming errors as well as malicious actors.

A key goal of e-voting research is minimizing the trust assumptions needed for security [1]–[7]. This is especially crucial for verifiability which should, ideally, allow a voter to check whether the result is correct and takes their own vote into account even in the worst case where all parties running the election act maliciously and collude with each other. One common trust assumption that has garnered interest concerns the bulletin board (BB) component.

E-Voting with BBs. A BB is supposed to act as a shared broadcast channel with memory [8]. It consists of one or more servers that maintain the internal memory/state, i.e., a list of client inputs that have been added successfully to the BB. A secure implementation of a BB is typically expected to provide at least *consistency (of outputs)* [9], [10]. That is, the internal memory list is append-only and clients reading from the BB always obtain prefixes of that list.

Most e-voting systems rely on a BB to share and publish data among participants and external observers, including election parameters, encrypted votes, the election result, and verification data such as zero-knowledge proofs (ZKPs). Security analyses of e-voting systems, such as [11]–[18], commonly assume that the underlying BB is not just trusted in the sense that it always provides consistent outputs from an append-only immutable state. They rather use the much stronger assumption of the BB being *perfect* [5], [10], at least in some parts/aspects, such that additionally (i) all outputs read by clients are not just consistent but even identical, (ii) all inputs will always end up in the internal memory, i.e., inputs cannot be dropped, (iii) new inputs are immediately added to the internal memory and then readable by everyone without delay, and/or (iv) items in the BB are ordered chronologically by when clients submitted them. Such BBs that are not just trusted but in some parts or even fully perfect cannot exist in reality. In an actual implementation of a BB running in an asynchronous unreliable network, called a *real BB* in what follows, inputs might get lost during transmission and never end up in the internal memory, inputs take time to become available as outputs and might change their order in the process, and outputs received at the same time by clients might not be identical but only consistent as they might have been generated and sent by servers at different points in time. Malicious servers in a real BB can even *equivocate outputs* [5], that is, provide different clients with inconsistent outputs that are not prefixes of the same internal memory list. Such inconsistent outputs can contradict each other, e.g., by showing different sets of votes that were tallied in an election.

This gap is not just a theoretical concern. The assumption of a perfect BB rather precludes major attack classes available to real BBs [5], [7], [19], [20]. For example, Hirschi et al. [5] observe that a malicious real BB might be able to use equivocation to break verifiability, say, by dropping some votes while presenting each affected voter with a modified

view of the election that still includes their own vote. Voters performing verification might then mistakenly be convinced that the result took their vote into account when it was actually dropped. Hirschi et al. also show that several prominent e-voting systems, namely Helios [21], Belenios [22], and Civitas [23], are susceptible to such equivocation attacks on verifiability. In [20], Cortier and Lallemand observe that privacy can be compromised as well when real BBs are used. For example, the BB might selectively drop all but one vote of interest, which is then tallied and revealed as the result of the election. Notably, this issue can occur even when all BB servers are trusted as a network adversary can drop inputs sent by clients before even reaching servers.

Current State. Several works on provably secure e-voting address the trust assumption of perfect BBs. They can roughly be classified into two major approaches.

One approach aims to perform security analyses of e-voting systems directly based on real BBs rather than using (partly) perfect BBs as an abstraction, e.g., [4], [5], [9], [24]. This approach assumes a trusted threshold of servers – without a single trusted server, equivocation is trivial – which is much more realistic than a perfect BB. On a conceptual level, the work by Hirschi et al. [5] is particularly noteworthy. Among others, they identify fundamental classes of attacks on verifiability and propose new general verifiability notions for analyzing voting systems following this approach.

An alternative approach used by works such as [6], [7], [19] is to retain a (partly) perfect BB but to give the adversary additional capabilities to more closely resemble some of the capabilities of real BBs in asynchronous networks, e.g., by allowing an adversary to selectively drop or reorder inputs to the perfect BB or by giving the adversary full control over the BB but only during some phases of the election. Conceptually, the work by Cortier et al. [7] is one of the most important ones as they propose several new privacy notions for this approach.

Both approaches have in common that they cannot entirely eliminate the trust assumption on the BB, which is particularly troublesome for verifiability. In this work, we, therefore, *propose and explore a third approach* based on the notion of accountability, which, as we will formally prove for the first time, can be used to construct *secure e-voting systems based on entirely untrusted, hence possibly fully malicious, real BBs*.

Accountability. The works constructing or using real BBs mentioned above aim for so-called *preventive security* [25]. That is, it should be impossible to break a security property/goal of a BB, such as consistency, even if some servers running the BB are actively misbehaving. While very desirable, such preventive security guarantees generally necessitate trusting at least a threshold of servers.

In contrast to preventive security, the concept of (*individual*) *accountability* [25]–[28] intuitively ensures the following: if some intended security property/goal of a protocol, e.g., consistency in a BB, is violated, then one can obtain undeniable cryptographic evidence that identifies at least one misbehaving protocol participant that has deviated from the protocol. This

not only ensures that a security breach will be noticed – thus offering a type of verifiability [26] – but further enables holding parties accountable for misbehavior, e.g., via financial or contractual penalties, which strongly incentivizes malicious parties to follow the protocol honestly and not break security properties in the first place. As accountability does not preclude a breach of a security property – it only ensures that this will be observable and a culprit can be identified after the fact – it is possible to achieve accountability-based security even in a fully malicious setting without trusted parties (typically still requiring some cryptographic assumptions, e.g., a PKI). We note that accountability-based and preventive security are orthogonal concepts with different (dis-)advantages that can be used independently to protect a protocol. However, they can also be combined using accountability as a second layer of defense in case trust assumptions underlying preventive security are not met (see [27] for more details).

Accountability is widely used in many security fields, including auctions [29]–[31], secure multi-party computation (MPC) [32]–[37], public key infrastructures (PKIs) [38]–[40], and distributed ledgers [41]–[49]. Accountability has also been used in the area of e-voting [6], [21], [22], [26], [50], [51]. However, works that formally prove accountability of an e-voting system still assume an at least partly perfect BB. While accountability is often mentioned as desirable and sometimes also claimed for real BBs [9], [24], [52], only recently did Graf et al. establish the first provably secure BB called $\text{Fabric}_{\text{BB}}$ [10] which offers accountability w.r.t. consistency. That is, as long as there is no evidence of misbehavior, all clients will always receive consistent outputs. Notably, $\text{Fabric}_{\text{BB}}$ achieves this property without assuming any trusted servers, i.e., in a fully malicious setting. The security result is shown in a Universal Composability (UC) model. All guarantees thus hold even when $\text{Fabric}_{\text{BB}}$ is used as a subroutine within a higher-level protocol.

This work. As mentioned, our goal is to obtain provably secure E2E verifiable and private e-voting systems without trusted servers running the BB. The underlying idea is conceptually simple: Use a BB that provides accountability w.r.t. consistency even when all parties are malicious. The election verification procedure should then check whether any evidence of misbehavior in the BB was obtained and, if so, reject the election result. If no misbehavior is detected and, hence, consistency holds, verification proceeds to check the correctness of the election material stored on the BB.

While this approach has not been formally considered so far and no security notions or proofs for this setting exist, the idea itself has already been mentioned and was sometimes even used informally. For example, both Helios [21] and Belenios [22], which have been formally proven secure based on (partly) perfect BBs [6], [11], [12], observe that an actual BB implementation might misbehave and require that its outputs should thus be monitored/audited to detect such misbehavior. This is a type of accountability-based security. Considering such auditing only informally outside of a security analysis,

however, is not sufficient as illustrated by Hirschi et al. [5], who found several attacks on verifiability of Helios and Belenios that a malicious BB can perform without being detected by the auditing procedures suggested by those systems.

Investigating this approach more formally raises several questions that we address in this work such as: How do we define verifiability and privacy in a setting with a potentially fully malicious BB? Can we adapt or re-use previous notions for perfect and threshold-trusted BBs, at least to some extent? Is obtaining a provably secure e-voting system using accountable BBs even feasible? Are there inherent issues and limitations of this approach that protocol designers have to handle?

Contributions. We start the first formal investigation of the above idea and establish fundamental insights for this third approach. Our work is thus best seen as a counterpart to the foundational works of Hirschi et al. [5] and Cortier et al. [7], who established security notions and general results for the other two existing approaches. More specifically, our contributions are as follows:

E2E Verifiability: We observe that existing verifiability notions that assume perfect BBs, such as the ones studied in [18], and the new verifiability notions for threshold-trusted real BBs by Hirschi et al. [5] cannot be applied to voting systems with untrusted but accountable BBs. We, therefore, propose a novel E2E verifiability notion that is highly general: it applies not just to e-voting systems with accountable BBs but also, e.g., to those with perfect or threshold trusted real BBs.

Insights Into Individual and Universal Verifiability: Instead of E2E verifiability (“Does the election result correspond to the intended votes of all honest voters and at most one vote per dishonest voter?”), the literature often considers the combination of *individual verifiability* (“Can a voter check that their encrypted ballot is included in the set that was tallied?”) and *universal verifiability* (“Was the election result derived correctly from the set of encrypted ballots?”). We describe two novel attack classes on E2E verifiability that a malicious BB, depending on the voting system, might be able to perform without breaking individual or universal verifiability. Together with two related attack classes due to malicious BBs previously observed in [5], this shows that an E2E verifiability notion is necessary for analyzing the security of e-voting systems using real BBs; checking individual and universal verifiability is insufficient. This result supplements [6], [18] who observed that even for systems assuming a perfect BB splitting E2E verifiability into subproperties can be problematic.

Privacy: Existing privacy notions often hard-code a perfect BB into their definition, e.g., [11], [14]–[16], [53]. This includes the definitions proposed by Cortier et al. [7], which still assume a partly perfect BB that cannot equivocate. Some privacy notions are defined more generally, e.g., the one by Küsters et al. [54] is defined abstractly for arbitrary voting systems but so far has only been applied to systems where the BB is assumed to be at least partly perfect [50], [54]–[58]. We show that and how the definition by Küsters et al. can also be applied to analyze the privacy of e-voting systems

using a potentially malicious real BB. A major challenge is that untrusted BBs can always drop all but one vote, which will then be tallied and published, thus trivially breaking privacy of that vote [20], [55]. We address this impossibility by considering a class of risk-avoiding adversaries.

A Simple Proof-Of-Concept Voting System: As a case study, we construct a concrete e-voting system based on an arbitrary trustless BB that is accountable w.r.t. consistency. This system, called **ABOVE** (Accountable Bulletin Board-based Voting) follows the line of homomorphic aggregation-based systems such as Helios and Belenios [21], [22] and is purposefully kept simple to focus on identifying potential issues, limitations, and requirements inherent to this new approach. It should be seen as a proof of concept that includes only a minimal set of features and does not offer advanced mechanisms such as receipt-freeness [59], [60] or coercion-resistance [54], [60].

One of the main challenges in designing a secure system in this approach turns out to be asynchronous networks, including delays and message drops, that do not exist in perfect BBs. For example, even when all BB servers behave honestly, consistent outputs might still differ arbitrarily in length. A voting system, therefore, needs to provide stability under message extension to prevent attacks where, say, the election result could be changed by returning a longer or shorter sequence of messages. This property requires care due to new aspects not present or left implicit in settings with perfect BBs. Another example is the lack of reliability, i.e., BB inputs are not guaranteed to become part of the output. Among others, this rules out some standard constructions used in e-voting systems, such as the distributed threshold ElGamal key generation protocol proposed in [16], which requires a reliable broadcast channel to ensure secrecy of the key.

Feasibility Result: We show that ABOVE achieves both privacy and our new E2E verifiability notion, thus formally confirming for the first time that the long-standing idea of using accountability to protect against malicious untrusted BBs is indeed sound and applicable to the class of homomorphic aggregation-based e-voting systems. We show this result not for a specific BB but more generally for any BB accountable w.r.t. consistency in a UC model. As an immediate corollary, we obtain that this holds for Fabric_{BB} in particular. By this, we also establish the first concrete provably E2E verifiable and private e-voting system without assuming any trusted BB servers running in an asynchronous network.

In our verifiability proof, we bridge the gap between a game-based verifiability notion (defined as a trace property over protocol runs) and the UC security notion of the BB, which establishes computational indistinguishability of two systems but, in general, does not imply that arbitrary trace properties carry over from one system to the other (for interested readers we provide simple counterexamples in Appendix D). This uncommon combination of different styles of security definitions might be useful also for other works on e-voting.

Structure. We recall the definition of UC-secure accountable bulletin boards in Section II. Since the paper is easier to

follow after seeing a concrete example voting system, we start by describing and discussing ABOVE in Section III. In Sections IV and V, we define a new notion for and prove E2E verifiability. In Section VI, we study and prove privacy. Additional details are available in the appendix.

II. RECAP: UC-SECURE ACCOUNTABLE BBS

Universal composability (UC) is a simulation-based paradigm for modeling and proving the security of protocols (e.g., [61]–[63]). In a UC security analysis, one first defines an *ideal protocol/ideal functionality* $\mathcal{F}$ that specifies a protocol’s intended security and functional properties, i.e., $\mathcal{F}$ is secure by definition but typically cannot be run in reality. To then analyze the security of an actual protocol $\mathcal{P}$, the *real protocol*, one shows that there exists a probabilistic polynomial-time (ppt) ideal adversary/simulator $\mathcal{S}$ controlling the network of $\mathcal{F}$ such that no ppt distinguisher $\mathcal{E}$, called *environment*, can tell whether it is running in the *real world* with $\mathcal{P}$ or in the *ideal world* with $\mathcal{F}$ and $\mathcal{S}$ (written $\mathcal{F} \mid \mathcal{S}$). Since $\mathcal{F}$ is secure by definition, the real protocol $\mathcal{P}$ must, therefore, be at least as good. One also says that $\mathcal{P}$ is a secure implementation of or (UC-)realizes $\mathcal{F}$, written $\mathcal{P} \leq \mathcal{F}$.

UC models provide a so-called composition theorem which states that, if $\mathcal{P} \leq \mathcal{F}$, then a ppt higher-level protocol $\mathcal{Q}$ using $\mathcal{P}$ as a subroutine (written $\mathcal{Q} \mid \mathcal{P}$) realizes $\mathcal{Q}$ using $\mathcal{F}$ (i.e., $\mathcal{Q} \mid \mathcal{P} \leq \mathcal{Q} \mid \mathcal{F}$). That is, one can first analyze $\mathcal{Q}$ w.r.t. the simpler $\mathcal{F}$, typically showing that $\mathcal{Q} \mid \mathcal{F}$ realizes some other ideal functionality $\mathcal{F}'$, and then replace $\mathcal{F}$ by $\mathcal{P}$ without further proof.

An Ideal Functionality for Accountable BBS. To securely access data stored in a real BB run by servers, higher-level protocols generally have to run a specific BB client software as a subroutine. For example, clients might have to use server public keys to verify signatures on outputs before returning them to higher-level protocols, might have to query multiple servers and combine a threshold of responses to obtain an overall output, or might have to sign inputs from higher-level protocols due to an access control mechanism. In [10], Graf et al. therefore defined an ideal bulletin board functionality $\mathcal{F}_{\text{BB}}$ that abstractly captures not just BB servers but also clients while formalizing accountability w.r.t. consistency for outputs returned by clients to higher-level protocols.

Intuitively, $\mathcal{F}_{\text{BB}}$ contains several BB clients that offer an input/output interface for the environment/higher-level protocols such as e-voting systems. The environment can send an input (write, m) to a client to request that it tries to add the message m to the BB. The environment can also send an input read to a client to request that it tries to read the current state of the BB and return the result as an output. Whenever a client in $\mathcal{F}_{\text{BB}}$ receives such input, $\mathcal{F}_{\text{BB}}$ forwards it to the network adversary, who can then decide whether and how to proceed. Notably, an adversary can decide to drop or delay the result of an input, and they can also choose the output for successful read requests subject to some restrictions explained below. Internally, $\mathcal{F}_{\text{BB}}$ keeps an ordered append-only sequence of messages msglist . The adversary can extend

msglist in arbitrary ways at any time, e.g., by adding a message m that was part of an input (write, m) received by a client or by adding new messages of their choosing. Finally, the adversary can corrupt clients and servers at any time to gain full control over them.

To define accountability of security properties, $\mathcal{F}_{\text{BB}}$ includes a so-called judge $J_{\text{BB}}^{\text{acc}}$. Unlike clients, $J_{\text{BB}}^{\text{acc}}$ does not model a dedicated party but rather abstractly captures an auditing algorithm; this is a common modeling technique for formalizing accountability and verifiability [18], [26], [27]. In a realization $\mathcal{P}_{\text{BB}}$ of $\mathcal{F}_{\text{BB}}$, the implementation of $J_{\text{BB}}^{\text{acc}}$ specifies the exact data and algorithm used to detect misbehavior, possibly resulting in a *verdict* that identifies a malicious party that can be held accountable. Here, we consider only realizations of $J_{\text{BB}}^{\text{acc}}$ that run on publicly available data such that any party can run the judging algorithm in practice. Hence, such a judge represents arbitrary parties that try to check whether consistency holds; such parties can be BB clients or servers, parties from a higher-level protocol, or outside observers.

Accountability w.r.t. consistency is then formalized by $\mathcal{F}_{\text{BB}}$ as follows. As long as the judge $J_{\text{BB}}^{\text{acc}}$ in $\mathcal{F}_{\text{BB}}$ has not yet computed a verdict, then whenever the adversary instructs an honest client to return an output r to a read request, r is a prefix of msglist . Conversely, once $J_{\text{BB}}^{\text{acc}}$ has identified at least one malicious party in a verdict, the adversary can return arbitrary, possibly contradictory, outputs r . The environment/higher-level protocols can query $J_{\text{BB}}^{\text{acc}}$ to obtain the current verdict, if any. This captures that a party (possibly from the higher-level protocol) can, at any point in time, decide to run the judging algorithm of a realization of $\mathcal{P}_{\text{BB}}$ on currently available public data to detect misbehavior. If there is no verdict yet, then consistency holds.

Observe that from this definition of $\mathcal{F}_{\text{BB}}$, it follows that any realization $\mathcal{P}_{\text{BB}}$ provides accountability w.r.t. consistency: Assume that the implementation of the judging algorithm $J_{\text{BB}}^{\text{acc}}$ in $\mathcal{P}_{\text{BB}}$ does not detect misbehavior and hence does not return a verdict. Then, by indistinguishability, the ideal judge in $\mathcal{F}_{\text{BB}}$ also does not return a verdict. In this case $\mathcal{F}_{\text{BB}}$ guarantees that the outputs of honest clients are consistent. Again, by indistinguishability, all outputs of honest clients in the realization $\mathcal{P}_{\text{BB}}$ are thus also consistent.

UC-Secure Accountable BBS Exist. This work is built on, and all our results are shown for an arbitrary real BB $\mathcal{P}_{\text{BB}}$ that realizes $\mathcal{F}_{\text{BB}}$. Recently, Graf et al. [10] constructed the first such BB called $\text{Fabric}_{\text{BB}}$. This BB is a slight extension of the prominent Hyperledger Fabric protocol [64] with an efficient implementation. Graf et al. showed:

Theorem 1 (Security of $\text{Fabric}_{\text{BB}}$, informal). *It holds true that $\text{Fabric}_{\text{BB}} \leq \mathcal{F}_{\text{BB}}$. This holds in an asynchronous network, even if all BB servers and arbitrary clients are malicious.*

We note that the technical details of $\text{Fabric}_{\text{BB}}$ are irrelevant to this work since all results are shown using only that $\text{Fabric}_{\text{BB}} \leq \mathcal{F}_{\text{BB}}$. For the details of $\text{Fabric}_{\text{BB}}$, refer to [10].

III. A SIMPLE E-VOTING SYSTEM BASED ON A TRUSTLESS BB

In Section III-A, we describe our proof of concept system ABOVE, which follows a standard homomorphic aggregation-based design in the style of Helios, Belenios, and similar systems [21], [22], [50] but uses an accountable trustless BB. In Section III-B, we highlight and discuss important properties necessary for building securely on such a BB.

A. System Description

Building blocks. ABOVE uses the following standard cryptographic primitives: Full threshold exponential ElGamal encryption $\mathcal{E} = (\text{Enc}, \text{Dec})$ where n_{talliers} parties each hold a share of the private key. An EUF-CMA-secure *signature scheme* $\mathcal{S}$ for authentication. A *non-interactive zero-knowledge proof* (NIZKP) $\pi^{\text{KeyShareGen}}$ for proving knowledge and correctness of a private decryption key share; a NIZKP π^{Enc} for proving knowledge and correctness of a plaintext vector contained in a ciphertext vector; a NIZKP π^{DecShare} for proving that a private decryption key share was correctly applied to a given ciphertext (see, e.g., [11] for possible NIZKPs. We recall formal security definitions of the required cryptographic primitives in Appendix C). We also assume a secure PKI.

We use an arbitrary (possibly trustless) real BB protocol $\mathcal{P}_{\text{BB}}$ that realizes $\mathcal{F}_{\text{BB}}$. Note that while $\mathcal{P}_{\text{BB}} \leq \mathcal{F}_{\text{BB}}$ implies that as long as no misbehavior is detected all outputs remain consistent, such a $\mathcal{P}_{\text{BB}}$ does not necessarily prevent an adversary from, e.g., dropping, reordering, or delaying inputs arbitrarily. It might also happen that outputs returned to different parties, while consistent, differ in length. All of this can simply be due to an asynchronous, unreliable real-world network. Dealing with these potential issues is the main challenge in designing a voting system on top of such a BB (see also the discussion in Section III-B).

Protocol participants. ABOVE is run among an election authority Auth, voters $V_1, \dots, V_{n_{\text{voters}}}$,¹ and talliers $T_1, \dots, T_{n_{\text{talliers}}}$. All parties also act as clients in $\mathcal{P}_{\text{BB}}$ by running the BB client code as a subroutine. We say “publish a message m on $\mathcal{P}_{\text{BB}}$ ” to mean that a party runs the client code to submit m as input, which then may or may not end up in the BB state; analogously for “read/obtain some data from $\mathcal{P}_{\text{BB}}$ ”. While we treat $\mathcal{P}_{\text{BB}}$ as a black box, further internal parties/servers are typically running the BB.

The election authority is responsible for determining trusted setup parameters but is not involved in evaluating the election; it is thus, besides the PKI, the only trusted entity. The talliers will tally the voters’ ballots. In order to avoid that a small set of malicious talliers learns how each voter voted, we distribute the secret decryption key among them so that all n_{talliers} talliers must collaborate to decrypt ballots.

The election authority, all voters, and all talliers own a signing key pair for $\mathcal{S}$ to authenticate their messages. To publish complaints when voters detect that their vote was

dropped, we require that voters can use a separate reliable channel, i.e., a public channel without message loss; all other communication is via unreliable networks/the unreliable $\mathcal{P}_{\text{BB}}$ (see the discussion in Section III-B).

Overview. There are four protocol phases: *Setup*, *voting*, *tallying*, and *public result verification*. During tallying, votes are aggregated homomorphically before decryption to protect the privacy of individual votes. Voters can run a *vote verification* algorithm to check whether their vote was counted or dropped.

Setup phase. The election authority Auth determines the parameters of the current election, including i) the ElGamal group $\mathbb{G}$ of size p with generator g , ii) the list of eligible voters $\text{id}_{\text{voters}}$ where each voter V_i is identified by a unique ID idv_i and their public signing key $\text{pk}_{V_i}^{\mathcal{S}}$, iii) the list of talliers identified via an ID idt_i and their public signing key $\text{pk}_{T_i}^{\mathcal{S}}$, and iv) the set $C \subseteq \{0, \dots, n_{\text{vpc}}\}^{n_{\text{cand}}}$ of valid non-abstention votes,² called *choice space*, where n_{cand} denotes the number of candidates and n_{vpc} is an upper bound for the number of votes that a voter may assign to each candidate. For example, in an election with three candidates where voters can assign a single vote to one candidate, we set $C = \{(c_1, c_2, c_3) \mid c_i \in \{0, 1\}, \sum_{i=1}^3 c_i = 1\}$. Auth signs and then publishes the parameters on $\mathcal{P}_{\text{BB}}$.

All talliers T_i locally choose a private ElGamal encryption key share $\text{sk}_i^{\mathcal{E}} \leftarrow^{\$} \mathbb{Z}_p$ uniformly at random, compute a corresponding public key share $\text{pk}_i^{\mathcal{E}} = g^{\text{sk}_i^{\mathcal{E}}}$, and create a NIZKP $\pi_i^{\text{KeyShareGen}}$ to prove knowledge of $\text{sk}_i^{\mathcal{E}}$ corresponding to $\text{pk}_i^{\mathcal{E}}$. Each tallier T_i then signs and publishes the message $(\text{idt}_i, \text{pk}_i^{\mathcal{E}}, \pi_i^{\text{KeyShareGen}})$ on $\mathcal{P}_{\text{BB}}$.

A protocol participant considers the setup finished if, after reading from $\mathcal{P}_{\text{BB}}$, they see the election parameters and one key share with valid signature per tallier (if there are multiple messages by the same signer, then all but the first are ignored). Note that this view can differ for each reader since $\mathcal{P}_{\text{BB}}$ might not (yet) have delivered some of the setup messages. After the setup is finished, the protocol participant can check the ZKPs and, if they are valid, compute the overall public encryption key $\text{pk}^{\mathcal{E}} = \prod_{i \in \{1, \dots, n_{\text{talliers}}\}} \text{pk}_i^{\mathcal{E}}$ of the election. In that case this protocol participant considers the system ready for voting. Otherwise, $\text{pk}^{\mathcal{E}}$ is undefined, and voting is not possible.

Voting phase. A voter V_i can abstain or cast a well-formed vote $\mathbf{v} \in C$. In the latter case, they wait until they can obtain $\text{pk}^{\mathcal{E}}$ from their output of $\mathcal{P}_{\text{BB}}$ and then encrypt each entry $\mathbf{v}[j]$ by computing an ElGamal ciphertext $\text{ct}_i[j] := (g^r, g^{\mathbf{v}[j] \cdot (\text{pk}^{\mathcal{E}})^r})$ with $r \leftarrow^{\$} \mathbb{Z}_p$. This results in a ciphertext vector ct_i , where each entry encrypts the number of votes assigned by V_i to candidate j . Voter V_i also computes a NIZKP π_i^{Enc} to prove that they know which vote $\mathbf{v}$ the vector ct_i encrypts and that $\mathbf{v} \in C$. Finally, V_i signs their ballot $b_i = (\text{idv}_i, \text{ct}_i, \pi_i^{\text{Enc}})$ and publishes the signed ballot on $\mathcal{P}_{\text{BB}}$. They also store the signed ballot locally for verification.

If a voter never sees the setup phase finish, they sign a complaint $(\text{idv}_i, \text{UnableToVote})$, indicating that their vote

¹Since ABOVE is a proof-of-concept system to establish viability, we simplify by not distinguishing between voters and their voting devices.

²We use a special vote $\text{abstain} \notin C$ to describe abstention. $\tilde{C} := C \cup \{\text{abstain}\}$ denotes the set of all valid votes, including abstention.

was dropped, and publish their complaint via their reliable channel for complaints.

The election authority Auth determines when voting closes and then publishes a signed message `VotingClosed` on $\mathcal{P}_{BB}$. Just as for the setup phase, the view on whether the voting phase has finished might differ for each reader of $\mathcal{P}_{BB}$.

Vote verification. A voter V_i who submitted a signed ballot b_i can verify that their vote v will be counted by reading the current list of ballots from $\mathcal{P}_{BB}$ and checking that (i) b_i (including the signature) appears in the output and (ii) there is no valid signed `VotingClosed` message before b_i . If this fails (and the voter no longer wants to wait for any possibly delayed messages from $\mathcal{P}_{BB}$), they sign a complaint (`idvi`, `VoteVerifFailed`) and publish it via their reliable channel for filing complaints.

Vote verification can be performed at any point in time, also immediately after submitting a vote, which conforms to the idea of “Vote&Go” [5], [65]. This improves the probability p_{verif} that a voter performs vote verification which is needed to detect dropped votes (cf. Section III-B).

Tallying phase. Once a tallier T_i reading from $\mathcal{P}_{BB}$ sees that the voting phase has finished, T_i starts tallying as follows.

1. *Homomorphic aggregation.* Let $\mathbf{b}$ be the ordered list of all ballots for the current election contained in T_i ’s output of $\mathcal{P}_{BB}$ after the election setup has finished with a well-defined $\text{pk}^\mathcal{E}$ and before the first signed `VotingClosed` message. T_i then removes ballots (i) that do not contain the ID `idvj` of an eligible voter, (ii) where the signature is invalid under the public key $\text{pk}_j^\mathcal{S}$ corresponding to the ID `idvj` in the ballot, or (iii) where the NIZKP is not valid. Then, T_i removes all but the first ballot for any given `idvj` such that, for each voter, at most one vote is counted. Finally, T_i performs ballot weeding, i.e., removes all ballots from $\mathbf{b}$ that are duplicates w.r.t. the ciphertext vector ct only keeping the first one. Ballot weeding protects against *replay attacks* which jeopardize vote privacy [66], [67]. From the resulting set of valid ballots $\mathbf{b}_{\text{valid}}$, if it is non-empty, T_i then homomorphically aggregates all vectors ct_j entrywise to obtain a vector ct_{aggr} where each entry is of the form $\text{ct}_{\text{aggr}}[j] = (g^{\sum_i r_i}, g^{\sum_i v_i[j]}) \cdot (\text{pk}^\mathcal{E})^{\sum_i r_i}$.

2. *Partial decryption.* T_i locally computes a vector of partial decryptions ds_i by applying their secret key share $\text{sk}_i^\mathcal{E}$ to each entry of ct_{aggr} . That is, for $\text{ct}_{\text{aggr}}[j] = (h_{1,j}, h_{2,j})$, the tallier computes $\text{ds}_i[j] = (h_{1,j})^{\text{sk}_i^\mathcal{E}}$. They further generate proofs $\pi_{i,j}^{\text{DecShare}}$ for each partial decryption $\text{ds}_i[j]$, showing that it was computed using secret witness $\text{sk}_i^\mathcal{E}$. T_i combines ds_i and all proofs into a message, signs it, and publishes the signed message on $\mathcal{P}_{BB}$.

Computing the result. Everyone can compute the election result by reading from $\mathcal{P}_{BB}$: Start by recomputing ct_{aggr} in the same way as talliers do. If the output returned by $\mathcal{P}_{BB}$ contains, after the (first) valid signed `VotingClosed` message, at least one valid signed partial decryption message with valid proofs $\pi_{i,j}^{\text{DecShare}}$ for each of the n_{talliers} many talliers, then take the first such message for each tallier to obtain partial decryption vectors ds_i . For each

entry/candidate j and $\text{ct}_{\text{aggr}}[j] = (h_{1,j}, h_{2,j})$, compute $h_{2,j} \cdot (\prod_{i \in \{1, \dots, n_{\text{talliers}}\}} \text{ds}_i[j])^{-1} = g^{\sum_i v_i[j]}$. The votes $\sum_i v_i[j]$ for candidate j can then be obtained by computing the discrete logarithm; this is feasible since the set of possible exponents is small in this case.

If the above fails, e.g., because in the output read from $\mathcal{P}_{BB}$ the setup or voting phases have not yet concluded, the setup failed and $\text{pk}^\mathcal{E}$ is undefined, or there are not (yet) valid partial decryption messages for all talliers, then there is no election result (an error election result). This, again, depends on the party reading from $\mathcal{P}_{BB}$ and might be a temporary error.

Election verification / E2E verifiability. By reading from $\mathcal{P}_{BB}$ and given the published voter complaints, everyone can verify that the election result indeed corresponds to the will of the voters. We define the exact procedure as part of our verifiability proof in Section V.

B. Discussion

Reliable channels for complaints. All e-voting systems, irrespective of the type of BB they use, have to deal with the following general issue: if the tally is missing a ballot from some voter V_i , then only V_i themselves can determine whether this is correct, i.e., whether they intended to abstain or their ballot was dropped maliciously. A standard approach to solve this issue is to require that voters verify whether their ballot was dropped and, if so, notify election authorities. Suppose an adversary can drop or hide these notifications/complaints. In that case the adversary can break E2E verifiability because they can drop the corresponding votes without the public noticing that the election result is incorrect.

Reliable channels for complaints are thus inherently required for verifiable elections, yet this requirement often has remained implicit or is covered up by much stronger assumptions. In works that consider individual and universal verifiability (e.g., [2], [12]), handling failures of vote verification is typically considered out of scope of the cryptographic protocol. While analyzing E2E verifiability requires treating complaints explicitly as part of the protocol – at least for systems where submitted ballots might get dropped – works such as [50] post complaints on a perfect BB that already offers much stronger properties. This, however, is insecure when a BB might misbehave or networks are asynchronous. E-Voting systems based on real BBs thus have to explicitly require a separate reliable channel for E2E verifiability.

This is still a drastic improvement over assuming a perfect or a real but reliable BB. Not only does this assumption cover far fewer messages. It is also not necessary that complaints are consistent, ordered in a specific manner such as chronologically, or immediately visible to everyone. In practice, a voter could, e.g., send their complaint to many different servers that provide it to anyone verifying the election but that, unlike real BBs, do not have to interact and synchronize with each other; the same server does not even have to provide consistent sets of complaints to different users. As long as the complaint is not dropped on at least one route,

this is sufficient for security. We also note that our E2E verifiability analysis (Section V) can easily be adapted to a weaker assumption where channels are somewhat unreliable such that a certain number of complaints might get dropped, weakening verifiability accordingly, cf. Footnote 6.

Stability under message extension. Since even consistent outputs of $\mathcal{P}_{BB}$ can still differ in length, it is crucial that once a property of the election system - such as the public election key, a change of two phases, the election result, or the ballot that is counted towards the result for a specific user - can be determined from an output, then any longer output will lead to the same property. Otherwise, verifiability can break down in various ways. For example, if a tallier can later change the public encryption key or the point in time when setup was concluded, say, by submitting a second setup message with a different public key share, then this would retroactively invalidate already submitted and verified votes. This is a security critical point that has to be considered in specifications, unlike for systems using perfect BBs.

Further subtle differences compared to using a perfect BB. Voting systems building on top of perfect BBs might optionally use but do not need explicit messages to indicate the start of a new protocol phase. This is because readers of such BBs are synchronized by always seeing the full and current state of the BB. However, for a real BB where voters might not see all stored messages, an explicit `VotingClosed` message or an equivalent mechanism becomes mandatory. Without this, it could happen that vote verification succeeds because the voter finds their ballot at, say, position 100 of an output of $\mathcal{P}_{BB}$. However, it is not counted because $\mathcal{P}_{BB}$ might show only the first 99 votes to talliers when they start tallying.

Also, if election parameters are distributed via a real BB, then the BB or even just a network adversary can prevent voters from running the voting algorithm, effectively dropping their vote, by never delivering necessary data such as the public encryption key. By the same reasoning as for submitted but dropped ballots, handling this case in verification requires a complaint `UnableToVote`. Note that this type of complaint is not needed in systems using reliable perfect BBs.

IV. E2E VERIFIABILITY FOR MALICIOUS BBs

In this section, we observe that we cannot apply existing verifiability notions for analyzing e-voting systems where the BB can misbehave arbitrarily – including accountable BBs as a special case. We hence propose a new, more general E2E verifiability notion by instantiating the *KTV verifiability framework* [26], a general framework for designing new verifiability notions for arbitrary types of protocols. KTV appears to be the best choice for this task since it covers standard e-voting verifiability notions as instances [18] and forms the basis of the new notions for threshold trusted BBs proposed in [5].

Our insights and results complement and extend those by Hirschi et al. [5], who previously studied verifiability notions in the presence of misbehaving BBs, observed that existing notions could not be applied, and proposed a different solution

that, however, is not applicable to accountable BBs. We compare both works in Section IV-D.

We start by briefly recalling the KTV framework including existing verifiability notions for e-voting systems.

A. Recap: The KTV Verifiability Framework

Judges. Just as for accountability in Section II, the KTV verifiability framework uses the concept of a *judge* J to formalize the (*judging*) *algorithm/procedure* used to verify the correctness of a protocol run. Intuitively, its task is to detect whether a desired *security goal*, such as the correctness of an election result, has been violated and output `reject` in this case; otherwise, it may output `accept`. To make this decision, the judge might *collect or receive some data as evidence* and *performs certain checks*, such as the verification of zero-knowledge proofs, that depend on the protocol specification. For e-voting protocols, the input to the judge typically consists solely of public information, i.e., in our case, the information posted on the bulletin board and complaints issued by voters via their public reliable channels. As in Section II, one can thus think of J as a “virtual” entity that only exists for modeling purposes. In reality, the judging procedure can be carried out by any party, including external observers and also voters themselves, who have access to the same evidence.

Goals. The KTV verifiability framework is centered around the notion of a security goal of a protocol $\mathcal{P}$. Formally, a goal γ is simply a set of protocol runs that contains exactly those runs that are “correct” in some protocol-specific sense. Specifying a new verifiability definition via the KTV framework mainly entails defining a suitable goal.

Cortier et al. [18] expressed and compared several established verifiability notions for e-voting systems as KTV goals. They conclude by recommending the following goal $\gamma_k(\phi)$ for defining and analyzing the E2E verifiability of e-voting systems. Here, ϕ is a boolean formula describing which combinations of protocol participants are assumed to be honest for verifiability.³ Intuitively, $\gamma_k(\phi)$ is met if the election result can be obtained with at most $k \in \mathbb{N}$ modifications (e.g., dropping votes) to the intended votes of honest voters. The parameter k allows for computing and comparing how likely different numbers of vote manipulations might go undetected by verification. For example, some voters might not check that their ballot actually ended up in the BB, giving an adversary a non-zero chance δ to drop those ballots without this being detected. However, in a good voting system, δ should decrease the more votes are dropped, i.e., the higher k . This goal and close variants have been used to analyze several e-voting systems (see, e.g., [6], [26], [50], [54], [56]).

Formally, $\gamma_k(\phi)$ is defined using a distance function *dist*. This in turn is defined based on a counting function $f_{\text{count}}: \tilde{\mathcal{C}}^* \rightarrow \mathbb{N}^{\tilde{\mathcal{C}}}$ which, for a vector $\vec{v} = (v_1, \dots, v_l) \in \tilde{\mathcal{C}}^*$ (representing a multiset of valid votes including abstention; cf. Footnote 2), counts how many times each vote $v \in \tilde{\mathcal{C}}$ occurs

³A boolean formula captures static corruption. As described in [18], this can easily be generalized for arbitrary conditions on dynamic corruptions.

in $\vec{v}$. For example, for $\tilde{C}$ consisting of three options A, B , and C , we get $f_{\text{count}}(B, B, C) = (0, 2, 1)$, i.e., it assigns 0 to A , 2 to B , and 1 to C . By $f_{\text{count}}(\vec{v})[\mathbf{v}]$ we denote the $\mathbf{v}$ -th entry of $f_{\text{count}}(\vec{v})$, i.e., the number of times vote $\mathbf{v}$ was counted by $f_{\text{count}}(\vec{v})$ - e.g., $f_{\text{count}}(B, B, C)[B] = 2$. For two vectors of votes $\vec{v}_0, \vec{v}_1$, the distance function dist is then defined by

$$\text{dist}(\vec{v}_0, \vec{v}_1) = \sum_{\mathbf{v} \in \tilde{C}} |f_{\text{count}}(\vec{v}_0)[\mathbf{v}] - f_{\text{count}}(\vec{v}_1)[\mathbf{v}]|.$$

For example, $\text{dist}((B, C, C), (A, C, C)) = 3$.

Now, given a run r of a voting protocol, let $(\mathbf{v}_i \in \tilde{C})_{i \in \text{id}_{\text{voters}}^{\text{hon}}}$ be the vector of intended votes of the honest voters $\text{id}_{\text{voters}}^{\text{hon}}$ in r . Then, the goal $\gamma_k(\phi)$ is satisfied in r (i.e., r belongs to $\gamma_k(\phi)$) if one of the following conditions holds:

- (a) the honesty assumptions are not met (ϕ is false) since then the judge cannot be required to detect errors, or
- (b) ϕ holds true in r and there exist votes $(\mathbf{v}'_i \in \tilde{C})_{i \in \text{id}_{\text{voters}}^{\text{dishon}}}$ (representing possible votes of the dishonest voters $\text{id}_{\text{voters}}^{\text{dishon}}$ in r) and votes $\vec{v}_{\text{real}} = (\mathbf{v}_i^{\text{real}} \in \tilde{C})_{1 \leq i \leq n_{\text{voters}}}$ (representing votes of honest and dishonest voters that led to the published election result) such that:
 - (i) a (non-error) election result is published in r and this result is equal to the result that is computed from $\vec{v}_{\text{real}}$,
 - (ii) $\text{dist}(\vec{v}_{\text{ideal}}, \vec{v}_{\text{real}}) \leq k$,

where $\vec{v}_{\text{ideal}}$ is the concatenation of the vectors $(\mathbf{v}_i)_{i \in \text{id}_{\text{voters}}^{\text{hon}}}$ and $(\mathbf{v}'_i)_{i \in \text{id}_{\text{voters}}^{\text{dishon}}}$.

Verifiability. Once a goal γ is defined, this yields a corresponding verifiability definition following a simple idea: The judge J should accept a run only if the goal γ is met, except for an error probability of δ called *verifiability tolerance*. While $\delta = 0$ is desirable, as explained above there is usually a chance $\delta > 0$ for an attack to go unnoticed. The value of δ typically depends on but should decrease exponentially in the number of vote modifications k for good voting systems.

Formally, we say that a function $f : \mathbb{N} \rightarrow [0, 1]$ is δ -bounded if, for every $c > 0$, there exists η_0 such that $f(\eta) \leq \delta + \eta^{-c}$ for all $\eta > \eta_0$. Then:

Definition 1 (Verifiability [26]). *Let $\mathcal{P}$ be a protocol that includes a judge J , i.e., specifies a verification procedure. Let $\delta \in [0, 1]$ be a tolerance and γ be a goal. Then we say that the protocol $\mathcal{P}$ is (γ, δ) -verifiable by the judge/verification procedure J if for all ppt adversaries $\mathcal{A}$ running with the protocol $\mathcal{P}$ (written $(\mathcal{A} \mid \mathcal{P}) =: \pi$) the probability $\Pr[\pi(1^n) \mapsto \neg \gamma, (J : \text{accept})]$ that π running with security parameter η produces a run which is not in γ but mistakenly accepted by J is δ -bounded as a function of η .*

B. Verifiability in the Context of Malicious BBs

The goal $\gamma_k(\phi)$ as well as all other goals/corresponding definitions studied in [18] are well-defined only if, in runs where ϕ is true, (i) there exists a uniquely defined election result and (ii) there exists a uniquely defined set of eligible voters (which defines the sets $\text{id}_{\text{voters}}^{\text{hon}}$ and $\text{id}_{\text{voters}}^{\text{dishon}}$). Both requirements are easy to achieve by assuming a trusted BB

that guarantees to distribute the same view on this information to everyone. However, a real BB, including accountable ones, might misbehave and provide different contradicting views to different parties such that these goals cannot be applied. Notably, these requirements are not just technicalities of the definition of $\gamma_k(\phi)$ but preclude general classes of attacks on verifiability that a real BB might perform and which should be detected by a security notion:

As for (i), a malicious BB might collude with malicious talliers to show each voter different sets of ballots with correct decryption proofs for the corresponding aggregated ciphertexts such that each set yields a different election result. The election then fails at a fundamental level since it does not establish agreement on a result and should, therefore, also not succeed at verification. However, each voter would still be able to verify and hence be convinced that their own local election result is the correct one in the sense that their encrypted ballot was included in the set of ballots that they see and their result was computed correctly from this set of ballots. In practice, one can easily avoid this issue by broadcasting an official result outside of the BB, e.g., via television. However, if such a separate secure broadcast channel is necessary for security, then this should be detected by a good security notion.

As for (ii), if the BB distributes the set of eligible voters in a way that a malicious BB can send different sets to different parties, then it can again collude with malicious talliers to run the following attack: Let A, B, C be three voters who want to vote for X and let D, E be two voters who want to vote for Y . The BB shows A the set of eligible voters $\{A, D, E\}$, shows $\{B, D, E\}$ to B , and shows $\{C, D, E\}$ to everyone else including C, D, E . Now, even if the BB shows all submitted ballots to everyone, each voter would consider a different set of ballots to be eligible and, thus, to be tallied. If the BB provides different proofs of decryption depending on which ballots are expected to be tallied, it can convince all voters that their vote is tallied and that tallying was performed correctly. Yet, all parties agree on the incorrect result Y rather than X .

The above also establishes two novel gaps between the commonly used combination of individual+universal verifiability and full E2E verifiability: A malicious BB might convince voters that their vote is included in the tally (individual) and tallying was performed correctly (universal), yet this does not imply that there is a unique result that takes the honest votes into account (E2E). Together with two related but different gaps due to malicious BBs observed in [5] (see Section IV-D), this shows that an E2E verifiability notion is necessary to obtain meaningful security results when BBs might misbehave. Since this is still missing, we next propose the first such notion.

C. Generalizing E2E Verifiability

We generalize the goal $\gamma_k(\phi)$, thus obtaining a new E2E verifiability notion via Definition 1 that is applicable in a broader range of settings following a natural idea: We remove any runs from the goal where the election result or the set of eligible participants is not uniquely determined and identical for all honest parties that compute a result resp. read a

set. This adaptation makes the new generalized goal $\gamma_k^{\text{gen}}(\phi)$ well-defined even if the BB misbehaves. It also strengthens the corresponding verifiability notion as now the verification procedure/judge must additionally reject the election as invalid whenever the cases (i) and (ii) from Section IV-B occur, which captures that these are possible attacks that should be detected. We note that $\gamma_k^{\text{gen}}(\phi)$ does not require *all* parties to be able to compute/see both the set of eligible voters and the election result. Indeed, in a real-world unreliable network, it can always happen that, e.g., some parties have not yet received all outputs from the BB and hence cannot yet compute the verified result. Formally:

Definition 2 (General E2E Verifiability). *Let ϕ be a boolean formula describing honesty assumptions and let $k \in \mathbb{N}$. Let r be a run of an arbitrary voting system (as in Definition 1), which might include a malicious BB. Then r is in $\gamma_k^{\text{gen}}(\phi)$ if the honesty assumptions do not hold, i.e., ϕ is false, or the honesty assumptions are met and all of the following is true:*

- a) *The set containing all non-error election results computed by honest parties in r has size 1. That is, the election has produced a result, say result_r , it is unique, and all honest parties that computed a non-error result in r agree on it.*
- b) *The set containing all sets of eligible voters obtained by honest parties in r has size 1. That is, there is exactly one set of eligible voters in r , and all honest parties that see such a set also agree on it.*
- c) *Let $(\mathbf{v}_i \in \tilde{\mathcal{C}})_{i \in \text{id}_{\text{voters}}^{\text{hon}}}$ be the vector of intended votes of the honest eligible voters $\text{id}_{\text{voters}}^{\text{hon}}$ in r . Then there exist votes $(\mathbf{v}'_i \in \tilde{\mathcal{C}})_{i \in \text{id}_{\text{voters}}^{\text{dishon}}}$ for the dishonest eligible voters $\text{id}_{\text{voters}}^{\text{dishon}}$ and votes $\vec{\mathbf{v}}_{\text{real}} = (\mathbf{v}_i^{\text{real}} \in \tilde{\mathcal{C}})_{1 \leq i \leq n_{\text{voters}}}$ such that result_r equals the result computed from all votes in $\vec{\mathbf{v}}_{\text{real}}$, and $\text{dist}(\vec{\mathbf{v}}_{\text{ideal}}, \vec{\mathbf{v}}_{\text{real}}) \leq k$ where $\vec{\mathbf{v}}_{\text{ideal}}$ is the concatenation of $(\mathbf{v}_i)_{i \in \text{id}_{\text{voters}}^{\text{hon}}}$ and $(\mathbf{v}'_i)_{i \in \text{id}_{\text{voters}}^{\text{dishon}}}$.*

Discussion. While the main motivation for our new E2E verifiability definition based on the goal $\gamma_k^{\text{gen}}(\phi)$ are e-voting systems using fully untrusted but accountable BBs, Definition 2 is actually agnostic to the type of BB or whether a BB is used at all. It thus applies more generally to several other e-voting systems, including: (i) Systems based on perfect BBs where Conditions a) and b) in Definition 2 are trivially met in all runs and do not impose additional requirements on the judge. The goal $\gamma_k(\phi)$ is thus captured as a special case of $\gamma_k^{\text{gen}}(\phi)$. (ii) Systems based on real BBs that ensure (preventive security of) consistency assuming a trusted threshold of servers. (iii) Systems that establish agreement on eligible voters and/or the election result via a separate channel outside of a BB – say, by broadcasting the result on television – such that even a fully untrusted BB without accountability cannot present different results to different parties. (iv) Systems that do not use any BBs, e.g., because they are distributed protocols run among the voters themselves.

Conditions a) and b) mention honest parties that may or may not have obtained/computed the set of eligible voters/the election result. Typically, as for instance in ABOVE in

Section V-C, such parties include all voters, the (potentially external) parties running the verification procedure modeled by the judge, and any other parties with access to the BB.

Condition a) gives guarantees only for parties that successfully compute a (non-error) election result. It does not, however, require that *all* honest parties who run the election result computation are successful. This is because in asynchronous real world networks it can always happen that, say, Alice can compute a non-error election result while Bob at the same time might not (yet) see, e.g., all messages from the tallying phase such that he is unable to compute the result. For the same reason, Condition b) also gives guarantees only for those parties who succeed at reading a set of eligible voters.

Security analyses using previous verifiability notions based on perfect BBs only have to model runs of an e-voting system up to the point where talliers publish their final messages. Since Condition a) adds a statement about different parties computing a result from those messages – possibly at different points in time – a security analysis using Definition 2 should in addition also model arbitrary parties running the election result computation. We recommend letting the adversary decide who runs this computation and when. Since verifiability is shown for all adversaries, this then covers arbitrary combinations and scheduling of parties computing the election result.

Our goal $\gamma_k^{\text{gen}}(\phi)$ and the corresponding verifiability notion consider only a single judge. While one might expect that one has to consider multiple concurrent judges to define (E2E) verifiability in a setting where a malicious BB can provide different views to different parties running the same verification procedure, this is not necessary: If a single judge – representing an arbitrary but fixed party running the judging procedure on its local view of BB – can verify the election result, then the goal $\gamma_k^{\text{gen}}(\phi)$ implies that this result is correct, i.e., was obtained with less than k modifications to honest votes, and that all other honest parties who obtain an election result agree on this correct one. Conversely, if parties do not agree on a unique election result or the result is unique but incorrect, then no matter who runs the judge/which BB view is used, the judge will reject.

D. Comparison to Hirschi et al. [5]

Hirschi et al. also start from the verifiability definitions (expressed as KTV goals) in [18]. They observe that all of these definitions use perfect BBs providing identical outputs to all parties after the end of the election and are thus no longer well-defined for real BBs run by potentially malicious servers. Hirschi et al. then identify two general attack classes not covered by these definitions where a malicious BB shows different sets of ballots to different voters such that individual and universal verifiability checks succeed, yet all voters agree on the same but incorrect result, i.e., E2E verifiability is broken.

To solve well-definedness of previous verifiability notions in the presence of real BBs, Hirschi et al. propose a novel security notion for real BBs called *final consistency*. Intuitively, a BB achieves final consistency if it reaches a “final” state

where it stops accepting inputs after an election ends and then guarantees that, whenever a party receives a final state output from the BB, then this output will be identical to final state outputs received by all other parties. Identical final outputs are sufficient such that verifiability definitions based on perfect BBs, such as the ones studied in [18], remain well-defined when the perfect BB is replaced by a final consistent one. They therefore add the requirement to verifiability notions that the BB used in an e-voting system has the final consistency property. Hirschi et al. call these new notions *verifiability+* notions. They also show that real BBs with final consistency can be constructed from digital signatures and assuming a trusted threshold of servers running the BB. They further prove that verifiability results based on perfect BBs imply the corresponding verifiability+ version using a final consistent BB under some mild additional requirements.

The requirements and general attack classes (i) and (ii) that we identify in Section IV-B are closely related to the work of Hirschi et al. but new and distinct. Crucially, while finally consistent BBs are sufficient for previous verifiability definitions to remain well-defined, this requirement imposed by verifiability+ is stronger than requirements (i) and (ii) we identify. Verifiability+ cannot be applied to a wide range of voting systems such as those using fully untrusted BBs, systems using accountable BBs (as accountability does not preclude misbehavior such as different final outputs), systems using BBs that run continuously and never reach a final state, say, because they are shared with other elections, and systems without BBs such as distributed protocols run among voters.

In contrast, our general verifiability notion given in Definition 2, which is based on the less demanding and more abstract requirements (i) and (ii), does not mandate a specific BB – or even any BB at all. It is in fact applicable to all of the above cases. It further captures the verifiability+ variant of $\gamma_k(\phi)$ as a special case: Consider a system with a finally consistent BB. As long as the trust assumption for that BB holds, all readers obtain identical final outputs and hence will always compute the same set of eligible voters and election result from those outputs. The first two conditions of Definition 2 are thus always trivially met such that, just as in verifiability+, the judging procedure only has to verify the original $\gamma_k(\phi)$.

V. E2E VERIFIABILITY OF ABBOVE

In this section, we use our new E2E verifiability definition from Section IV-C to analyze the verifiability of ABBOVE.

A. Computational Model

We model ABBOVE using a general and established computational framework (see, e.g., [18], [26], [50], [54], [56]) which is based on the notion of a process. A process $\pi_{\mathcal{P}}$ modeling some protocol $\mathcal{P}$ is a set of interacting ppt Turing machines (ITMs, sometimes also called *programs*) that can send messages to each other via connected tapes. At any point in time, only one machine is actively running, namely, the most recent one to receive a message. The protocol $\mathcal{P}$ runs alongside an adversary $\mathcal{A}$, modeled via another process $\pi_{\mathcal{A}}$, which

controls the network and may corrupt protocol participants by sending a special *corrupt* signal via a connected tape to their ITMs. A corrupted ITM then acts as a pure message forwarder and thus gives full control to $\mathcal{A}$. Here, we only consider static corruption of parties. We write $\pi = (\pi_{\mathcal{P}} \mid \pi_{\mathcal{A}})$ for the combined process. We often use $\mathcal{P}$ and $\pi_{\mathcal{P}}$ interchangeably; the same for $\mathcal{A}$ and $\pi_{\mathcal{A}}$.

ABBOVE can be modeled straightforwardly as a protocol $\mathcal{P}_{\text{ABBOVE}}(n_{\text{voters}}, n_{\text{talliers}}, \mathcal{C}, \mu_{\text{vote}}, p_{\text{verif}})$ in the above sense. Recall from Section III that we denote the number of voters by n_{voters} , the number of talliers by n_{talliers} , and the set of valid votes by $\tilde{\mathcal{C}} := \mathcal{C} \cup \{\text{abstain}\}$. By μ_{vote} , we denote a distribution over $\tilde{\mathcal{C}}$ according to which each honest voter chooses their *intended vote*, and by $p_{\text{verif}} \in [0, 1]$ the probability that an honest voter chooses to verify their vote. (By this, we model that the adversary knows both distributions.)

In our model of ABBOVE, the trusted voting authority Auth is part of an additional incorruptible agent, the scheduler Sch. Besides playing the role of the authority, Sch schedules all other agents in a run according to the protocol. In particular, while the adversary can determine the order of activations of parties, Sch ensures that all honest voters have been activated at least once to (try to) vote before Auth closes the voting phase. Sch also ensures that all honest voters get activated at least once after their vote submission to decide whether they want to run vote verification. Sch further collects any complaints from voters and forwards them directly to the judge while also making them available to $\mathcal{A}$, which captures reliable public complaint channels. Besides Sch, we also model J as incorruptible. Note that this is not a trust assumption. It rather captures that the judge represents an arbitrary party running the judging algorithm on public data (see Section IV-A). An honest party can thus always choose to run the judging algorithm themselves to guarantee a correct execution without needing to trust a third party. We let the adversary decide when J runs election verification, which then produces an overall output *reject* or *accept* in that run.

Our model of ABBOVE includes an arbitrary accountable real BB $\mathcal{P}_{\text{BB}}$ defined as a UC protocol. This is possible because UC protocols are defined also using interacting ppt Turing machines. Since each party P of the voting protocol runs the client code of $\mathcal{P}_{\text{BB}}$, we formally represent a party P by two ITMs in our model: A higher-level program π_{hl} which runs the logic of the voting protocol and communicates via a directly connected tape with a subroutine $\pi_{\text{client-BB}}$ that runs the client logic of $\mathcal{P}_{\text{BB}}$.⁴ Since, in reality, both programs represent the same party and run on the same computer, we consider the corruption of one program to simultaneously corrupt the other.

B. Judge J

Next, we formally specify the judge J for ABBOVE. Since J defines the exact verification procedure used for verifying a run of the election, this is an integral part of the protocol specification of ABBOVE that we now complete.

⁴Interested readers might want to look at Figure 2 in Appendix A that illustrates the overall structure of our model.

The judge J has access to all published complaints of voters, which captures reliable complaint channels. It runs the accountability judging procedure J_{BB}^{acc} of $\mathcal{P}_{BB}$ as a subroutine. Verification then proceeds as follows:

- 1) Read from $\mathcal{P}_{BB}$ to obtain a sequence of messages $msgseq$ and run the judging procedure J_{BB}^{acc} as specified by $\mathcal{P}_{BB}$. If this detects a misbehaving party (blamed in a verdict), then J outputs `reject`. Otherwise, $msgseq$ is consistent with all outputs read by honest parties; the following computations are thus performed based on $msgseq$.
- 2) Try to compute the election result as per Section III. Recall that this involves, among others, computing the public encryption key from setup messages, recomputing the aggregated ciphertexts, and verifying signatures and NIZKPs. If this fails, i.e., the result is $\perp$, output `reject`.
- 3) Let id_{voters} be the set of eligible voters from the election setup message contained in $msgseq$ (if $msgseq$ does not contain this message or it is invalid, then the previous step already rejects since no election result can be computed). Check if there is a complaint with a valid signature signed by one of the voters in id_{voters} . If so, output `reject`.⁵
- 4) Otherwise, accept the election and its result.

C. E2E Verifiability Analysis of *ABOVE*

Our security result uses the following assumptions:

- (V1) π^{Enc} and $\pi^{DecShare}$ are NIZKPs and the signature scheme S is EUF-CMA-secure (we recall formal security definitions of cryptographic primitives we use in Appendix C).
- (V2) There is at most a negligible chance for two honest voters to generate the same ciphertext vector ct . E.g., for non-trivial ElGamal groups this is implied by the DDH assumption (cf. Footnote 11 on Page 22).
- (V3) $\phi := \text{hon}(\text{Sch}) \wedge \text{hon}(J)$.
- (V4) Voters have reliable channels to publish complaints outside the BB.

Theorem 2. *Let $\mathcal{F}_{BB}$ be an ideal BB functionality as described in Section II that provides at least accountability w.r.t. consistency, possibly in addition to further security or functional properties. Let $\mathcal{P}_{BB}$ be an arbitrary real BB protocol such that $\mathcal{P}_{BB} \leq \mathcal{F}_{BB}$. Let $\mathcal{P}_{ABOVE}(n_{voters}, n_{talliers}, C, \mu_{vote}, p_{verif})$ be the voting protocol based on $\mathcal{P}_{BB}$ as above and including the judge J from Section V-B. Let $k \in \mathbb{N}$. Under the assumptions (V1) to (V4), we have that $\mathcal{P}_{ABOVE}$ is $(\gamma_k^{gen}(\phi), \delta_k(p_{verif}))$ -verifiable where*

$$\delta_k(p_{verif}) = (1 - p_{verif})^{k+1}.$$

Proof sketch (see Appendix E for the complete proof). The proof consists of two parts. In the first part, we prove the result for a hybrid system $\pi_{ideal-BB}$ that works as $\mathcal{P}_{ABOVE}$ but replaces $\mathcal{P}_{BB}$ by $\mathcal{F}_{BB}$ and its simulator (cf. Figure 2). We perform several game hops using accountability w.r.t. consistency provided by $\mathcal{F}_{BB}$, standard arguments from

⁵To improve robustness against malicious unjustified complaints, the judge can alternatively reject only if there are at least $d+1$ complaints. This leads to a slightly worse verifiability level similar to what we explain in Footnote 6.

verifiability proofs of homomorphic aggregation-based systems using perfect BBs, and additional arguments to handle attacks based on unreliability and BB outputs differing in length (see also Section III-B). Intuitively, accountability w.r.t. consistency and stability under message extension are the main tools to show that if Conditions a) and b) of $\gamma_k^{gen}(\phi)$ are not met, then this is detected by the judge and the result is rejected. We are further able to show that the only way to change the election result – thus violating Condition c) – while remaining undetected with a non-negligible chance is by not including submitted ballots in the BB state before the `VotingClosed` message. As each dropped ballot changes the result by 1 and the affected voter will complain if they choose to verify their vote, we get an upper bound $(1 - p_{verif})^{k+1}$ of more than k modifications remaining undetected.⁶

In the second part, we show that the verifiability error for $\mathcal{P}_{ABOVE}$ (where $\mathcal{P}_{BB}$ is used) is negligibly close to the one for $\pi_{ideal-BB}$ (where $\mathcal{F}_{BB}$ is used). While one might expect this to trivially and always follow from the composability of UC security of $\mathcal{P}_{BB}$, this is, in general, not implied because verifiability is a trace property defined over runs of systems. Intuitively, this is because trace properties might not be computable for UC environments. E.g., a trace property can be defined over and depend on some internal behavior such as variables and messages that the UC environment cannot observe or a trace property might not be decidable in polynomial runtime. The probability for such trace properties can be vastly different between real and ideal world without contradicting UC security. For interested readers, we discuss this gap in more detail and provide two simple counter examples in Appendix D. Fortunately, we can show via a reduction that in our case the statement indeed follows. $\square$

Corollary 1. *Under the above assumptions and for the same goal and δ_k as in Theorem 2, $\mathcal{P}_{ABOVE}$ using Fabric_{BB} is $(\gamma_k^{gen}(\phi), \delta_k(p_{verif}))$ -verifiable.*

Proof. Follows by Theorems 1 and 2. $\square$

Since Theorem 2 only requires $\text{Fabric}_{BB} \leq \mathcal{F}_{BB}$, which holds true even when *all* parties running the BB are malicious by Theorem 1, this establishes, for the first time, *E2E verifiability of an e-voting system using a fully untrusted BB*.

VI. PRIVACY

We analyze the privacy of *ABOVE* using the established privacy definition by Küsters et al. in [54]. This definition allows for computing the exact privacy level that an e-voting protocol achieves based on factors such as the choice space C , the number n_{voters}^{hon} of (non-abstaining) honest voters, and the honest vote distribution μ_{vote} . Alternatively, the definition can also be used in the form of a more intuitive indistinguishability game. This game-based version has already been applied in, e.g., [50], [56] and is what we present and use here as well.

⁶If complaint channels are assumed to only be somewhat reliable such that up to d complaints might get dropped, then the verifiability error δ degrades to $\sum_{i=0}^d p_{verif}^i (1 - p_{verif})^{k+1-i}$.

A. Privacy Notion.

In a nutshell, the definition from [54] defines the privacy of an e-voting protocol as the inability of an adversary to distinguish whether an arbitrary but fixed honest voter v_{obs} (the *voter under observation*) voted for v_0 or v_1 . Here, we consider $v_0, v_1 \in C$, i.e., we do not require that a voting system hides whether v_{obs} abstained.

More formally, the privacy notion considers an e-voting protocol $\mathcal{P}$ with choice space C , a security parameter η , a ppt adversary $\mathcal{A}$, and a voter under observation v_{obs} . Given some vote $v \in C$, we denote by $(\pi_{v_{\text{obs}}}(v) \mid \pi_{\mathcal{P}}^* \mid \pi_{\mathcal{A}})$ the composition of v_{obs} 's program $\pi_{v_{\text{obs}}}(v)$ when taking v as their intended vote, all programs $\pi_{\mathcal{P}}^*$ of the remaining parties in $\mathcal{P}$, and the adversary's program $\pi_{\mathcal{A}}$. Note that the degree of how well the published aggregated election result hides v_{obs} 's vote, and hence the privacy of v_{obs} , depends on the number of honest voters that submit a vote. Conversely, abstaining honest voters do not improve privacy. In this section by $n_{\text{voters}}^{\text{hon}}$ we thus denote the number of non-abstaining honest voters, i.e., for those voters we consider an honest vote distribution μ_{vote} which does not return `abstain`. Honest voters who abstain are still captured but are subsumed in the remaining $n_{\text{voters}} - n_{\text{voters}}^{\text{hon}}$ voters, some of which might be malicious.

Now privacy is defined via the following *privacy game* $\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}})$:

$\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}})$:

- 1) $\mathcal{A}$ chooses two possible votes $v_0, v_1 \in C$ and sends them to the challenger.
- 2) The challenger chooses a bit $b \leftarrow \{0, 1\}$ uniformly at random, which determines v_{obs} 's vote v_b .
- 3) Run the protocol $(\pi_{v_{\text{obs}}}(v_b) \mid \pi_{\mathcal{P}}^* \mid \pi_{\mathcal{A}})$.
- 4) $\mathcal{A}$ guesses a bit $b' \in \{0, 1\}$.
- 5) If $b' = b$, return 1, otherwise return 0.

We denote by $\Pr[\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}}) \mapsto 1]$ the probability that $\mathcal{A}$ wins $\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}})$, where the probability is taken over the random coins used by the parties in $(\pi_{v_{\text{obs}}} \mid \pi_{\mathcal{P}}^* \mid \pi_{\mathcal{A}})$ as well as the random choice b by the challenger in step 2.

Privacy is then defined by comparing the winning probability of $\mathcal{A}$ attacking $\mathcal{P}$ with an ideal situation where an adversary/simulator $\mathcal{A}_{\text{ideal}}$ attacks an ideal voting protocol/functionality $\mathcal{P}_{\text{ideal}}$ that achieves the best privacy level by definition. More specifically, $\mathcal{P}_{\text{ideal}}(C, n_{\text{voters}}, n_{\text{voters}}^{\text{hon}}, \mu_{\text{vote}})$ is defined with the parameters as above. Intuitively, $\mathcal{P}_{\text{ideal}}$ computes an election result as an aggregation of v_{obs} 's vote v_b , $n_{\text{voters}}^{\text{hon}} - 1$ additional secret votes chosen independently according to μ_{vote} , and up to $n_{\text{voters}} - n_{\text{voters}}^{\text{hon}}$ votes from C but chosen by the adversary. $\mathcal{P}_{\text{ideal}}$ reveals only the election result but nothing else to the adversary. It therefore captures the best case where an adversary does not learn anything about v_b besides what it can compute from an election result which includes at most one copy of v_b and (at least) $n_{\text{voters}}^{\text{hon}} - 1$ independent secret votes.⁷ We recall the formal definition in

⁷This also rules out ballot stuffing/replay attacks where, e.g., v_b is submitted multiple times to inflate its influence on the result [66], [67].

Figure 3 in Appendix A.

Privacy is then defined and proven w.r.t. a specific set of adversaries, called “admissible adversaries” in what follows. We explain the purpose of this set and define it for **ABOVE** later. Formally:

Definition 3 (Privacy). *Let $\mathcal{P}$ be a voting protocol and let $\mathcal{P}_{\text{ideal}}$ be an ideal voting protocol.⁸ Then, $\mathcal{P}$ achieves (ideal) privacy, if for all admissible adversaries $\mathcal{A}$ on $\mathcal{P}$, there is an adversary $\mathcal{A}_{\text{ideal}}$ on $\mathcal{P}_{\text{ideal}}$ such that*

$$|\Pr[\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}}) \mapsto 1] - \Pr[\mathbb{E}^{\text{Priv}}(\mathcal{A}_{\text{ideal}}, \mathcal{P}_{\text{ideal}}, \eta, v_{\text{obs}}) \mapsto 1]|$$

is negligible as a function in η .

The strength of such a privacy result depends on the parameters chosen for $\mathcal{P}_{\text{ideal}}$. Importantly, a good privacy result should achieve a large value of $n_{\text{voters}}^{\text{hon}}$ as this determines how well the election result hides the vote of v_{obs} . Ideally, $n_{\text{voters}}^{\text{hon}}$ for $\mathcal{P}_{\text{ideal}}$ should be close to or identical to the number of non-abstaining honest voters in the real voting protocol $\mathcal{P}$. To understand the actual level of privacy - i.e., the attacker's success probability - offered by different instances of the ideal protocol $\mathcal{P}_{\text{ideal}}$, including how it varies with parameters like the choice space C , the number of honest voters $n_{\text{voters}}^{\text{hon}}$, and the vote distribution μ_{vote} , we refer interested readers to the analyses provided in [50], [54], [56].

B. Privacy in the Presence of Real BBs

As mentioned in Section I, an adversary $\mathcal{A}$ controlling a real BB or even just the network can trivially break privacy [6], [20], [50]: $\mathcal{A}$ might drop all but v_{obs} 's ballot which is then tallied and revealed as the result of the election. Formally, this means that in the presence of arbitrarily misbehaving adversaries, Definition 3 can only be shown for $\mathcal{P}_{\text{ideal}}$ with $n_{\text{voters}}^{\text{hon}} = 1$, i.e., when there is no secret vote in the result that hides the one of v_{obs} and, hence, v_{obs} 's vote is trivially revealed also in the ideal protocol.

In [7], Cortier et al. studied malicious BBs and found that, besides dropping inputs/votes, also re-ordering and modifying inputs can break privacy. In addition to these results, we note that equivocation enables further attacks on privacy. For example, suppose the BB distributes the public election key. In that case, the BB might provide different encryption keys to different voters. Among others, this can cause ballots to be successfully submitted yet not be counted towards the result: talliers seeing and using a different public key to verify the NIZKP π^{Enc} will consider π^{Enc} to be invalid w.r.t. their public key and hence drop the ballot. Crucially and unlike for ballots explicitly dropped by malicious parties, this way of removing a vote from the result would not be detected via vote verification by the affected voter who still finds their correct ballot in the output of the BB. In Appendix B, we describe another type of equivocation attack on privacy that affects systems using a decryption threshold $t < n_{\text{talliers}}$.

⁸Typically, $\mathcal{P}_{\text{ideal}}$ uses the same parameters as $\mathcal{P}$ except for potentially $n_{\text{voters}}^{\text{hon}}$. We discuss this in what follows.

By construction, for ABOVE the above types of attacks come at a high risk that rational adversaries would typically avoid. Firstly, since we use an accountable BB, i.e., $\mathcal{P}_{\text{BB}} \leq \mathcal{F}_{\text{BB}}$, one can use contractual penalties for detected misbehavior to incentivize malicious BBs to remain consistent which prevents equivocation attacks (but attacks based on consistent outputs that differ in length might still be possible). Secondly, for vote dropping, we have shown as part of proving Theorem 2 that, as long as the BB provides consistent outputs, the only ways for an adversary to drop/modify honest votes will lead to one of two complaints being published – in case of VoteVerifFailed with chance p_{verif} of running vote verification. Often, it is reasonable to assume that rational adversaries do not want to risk invalidating the entire election due to complaints, which might cause an investigation, just to reveal a single or a few votes. This observation motivated the notion of risk-avoiding adversaries in [50], [55] for which meaningful privacy results can be shown. Also other works such as [7] follow a similar idea by defining privacy only for adversaries that are unwilling/unable to risk a failure in vote verification or who do not learn the election result if there is a failure. In fact, in [20] Cortier and Lallemand showed that successful vote verification is necessary for privacy.

We therefore show privacy of ABOVE for the following class of rational adversaries. We call an adversary $\mathcal{A}$ *k-risk-avoiding* if the following two events (taken over all runs of $\mathcal{A}$ with ABOVE) have negligible probability: (i) The BB provides inconsistent views to two (or more) honest parties. (ii) If an output returned by the BB to an honest party contains a successful setup phase and a valid signed VotingClosed message (i.e., the output would be tallied by an honest tallier), then in that output and before the first VotingClosed message there are less than $n_{\text{voters}}^{\text{hon}} - k$ signed ballots submitted by honest voters. That is, $\mathcal{A}$ is willing to drop at most k ballots submitted by honest voters as $\mathcal{A}$ considers the probability of a complaint due to vote verification to be too high for larger numbers.

C. Privacy of ABOVE

We prove privacy using the following assumptions and class of adversaries:

- (P1) An adversary is admissible if it (i) does not corrupt Sch, (ii) corrupts at most $n_{\text{talliers}} - 1$ talliers, (iii) corrupts at most $n_{\text{voters}}^{\text{dishon}} = n_{\text{voters}} - n_{\text{voters}}^{\text{hon}}$ voters, and (iv) is *k-risk-avoiding*.
- (P2) The ElGamal group $\mathbb{G}$ is non-trivial and the DDH problem is hard in $\mathbb{G}$.
- (P3) $\pi_{\text{KeyShareGen}}$, π_{Enc} , and π_{DecShare} are NIZKPs and the signature scheme $\mathcal{S}$ is EUF-CMA-secure (we recall formal security definitions of the cryptographic primitives we use in Appendix C).

Now, we can show the following:

Theorem 3. *Consider ABOVE for choice space $\mathcal{C}$, number n_{voters} of voters, $n_{\text{voters}}^{\text{hon}}$ a lower bound for the number of honest non-abstaining voters, and vote distribution μ_{vote} for those voters. Then ABOVE achieves ideal privacy for*

$\mathcal{P}_{\text{ideal}}(n_{\text{voters}}, n_{\text{voters}}^{\text{hon}} - k, \mathcal{C}, \mu_{\text{vote}})$ against admissible adversaries in the sense of (P1) under Assumptions (P2) and (P3).

We give a detailed proof in Appendix F using a sequence of games that transforms the game $\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}})$ to the game $\mathbb{E}^{\text{Priv}}(\mathcal{A}_{\text{ideal}}, \mathcal{P}_{\text{ideal}}, \eta, v_{\text{obs}})$ for an ideal adversary $\mathcal{A}_{\text{ideal}}$. Theorem 1 then implies:

Corollary 2. *ABOVE using Fabric_{BB} achieves ideal privacy for the same parameters as in Theorem 3.*

While showing Theorem 3, we found a gap in the privacy proof of Helios given in [11] which is of independent interest. We provide details in Appendix B.

ACKNOWLEDGEMENTS

This research was partially funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation), Project-ID 459731562. We thank Mike Graf for helpful discussions and feedback.

REFERENCES

- [1] J. Benaloh, “Simple Verifiable Elections,” in *2006 USENIX/ACCURATE Electronic Voting Technology Workshop, EVT’06, Vancouver, BC, Canada, August 1, 2006*. USENIX Association, 2006.
- [2] V. Iovino, A. Rial, P. B. Rønne, and P. Y. A. Ryan, “Universal Unconditional Verifiability in E-Voting without Trusted Parties,” in *33rd IEEE Computer Security Foundations Symposium, CSF 2020, Boston, MA, USA, June 22-26, 2020*. IEEE, 2020, pp. 33–48.
- [3] J. Benaloh, M. Naehrig, and O. Pereira, “REACTIVE: Rethinking Effective Approaches Concerning Trustees in Verifiable Elections,” Cryptology ePrint Archive, Tech. Rep. 2024/915, 2024.
- [4] N. Chondros, B. Zhang, T. Zacharias, P. Diamantopoulos, S. Maneas, C. Patsonakis, A. Delis, A. Kiayias, and M. Roussopoulos, “D-DEMOS: A Distributed, End-to-End Verifiable, Internet Voting System,” in *36th IEEE International Conference on Distributed Computing Systems, ICDCS 2016, Nara, Japan, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 711–720.
- [5] L. Hirschi, L. Schmid, and D. A. Basin, “Fixing the Achilles Heel of E-Voting: The Bulletin Board,” in *34th IEEE Computer Security Foundations Symposium, CSF 2021, Dubrovnik, Croatia, June 21-25, 2021*. IEEE, 2021, pp. 1–17.
- [6] R. Küsters, T. Truderung, and A. Vogt, “Clash Attacks on the Verifiability of E-Voting Systems,” in *33rd IEEE Symposium on Security and Privacy (S&P 2012)*. IEEE Computer Society, 2012, pp. 395–409.
- [7] V. Cortier, J. Lallemand, and B. Warinschi, “Fifty Shades of Ballot Privacy: Privacy against a Malicious Board,” in *IEEE 33rd Computer Security Foundations Symposium, CSF 2020, 22-25 July, 2020*. IEEE Computer Society, 2020.
- [8] S. Hauser and R. Haenni, “Modeling a Bulletin Board Service Based on Broadcast Channels with Memory,” in *Financial Cryptography and Data Security - FC 2018 International Workshops, BITCOIN, VOTING, and WTSC, Nieuwpoort, Curaçao, March 2, 2018, Revised Selected Papers*, ser. Lecture Notes in Computer Science, vol. 10958. Springer, 2018, pp. 232–246.
- [9] A. Kiayias, A. Kuldmann, H. Lipmaa, J. Siim, and T. Zacharias, “On the Security Properties of e-Voting Bulletin Boards,” in *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy, September 5-7, 2018, Proceedings*, ser. Lecture Notes in Computer Science, vol. 11035. Springer, 2018, pp. 505–523.
- [10] M. Graf, R. Küsters, D. Rausch, S. Egger, M. Bechtold, and M. Flinspach, “Accountable Bulletin Boards: Definition and Provably Secure Implementation,” in *37th IEEE Computer Security Foundations Symposium (CSF 2024)*. IEEE, 2024, to appear.
- [11] D. Bernhard, O. Pereira, and B. Warinschi, “How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios,” in *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings*, ser. Lecture Notes in Computer Science, X. Wang and K. Sako, Eds., vol. 7658. Springer, 2012, pp. 626–643.

- [12] V. Cortier, C. C. Dragan, F. Dupressoir, and B. Warinschi, "Machine-Checked Proofs for Electronic Voting: Privacy and Verifiability for Belenios," in *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*. IEEE Computer Society, 2018, pp. 298–312.
- [13] W. Lueks, I. Querejeta-Azurmendy, and C. Troncoso, "VoteAgain: A scalable coercion-resistant voting system," in *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*. USENIX Association, 2020, pp. 1553–1570.
- [14] D. Bernhard, V. Cortier, D. Galindo, O. Pereira, and B. Warinschi, "SoK: A Comprehensive Analysis of Game-Based Ballot Privacy Definitions," in *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*. IEEE Computer Society, 2015, pp. 499–516.
- [15] D. Bernhard, V. Cortier, O. Pereira, B. Smyth, and B. Warinschi, "Adapting Helios for Provable Ballot Privacy," in *Computer Security - ESORICS 2011 - 16th European Symposium on Research in Computer Security, Leuven, Belgium, September 12-14, 2011. Proceedings*, ser. Lecture Notes in Computer Science, vol. 6879. Springer, 2011, pp. 335–354.
- [16] V. Cortier, D. Galindo, S. Glondu, and M. Izabachène, "Distributed ElGamal à la Pedersen: Application to Helios," in *Proceedings of the 12th annual ACM Workshop on Privacy in the Electronic Society, WPES 2013, Berlin, Germany, November 4, 2013*. ACM, 2013, pp. 131–142.
- [17] D. A. Basin, S. Radomirovic, and L. Schmid, "Alethea: A Provably Secure Random Sample Voting Protocol," in *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*. IEEE Computer Society, 2018, pp. 283–297.
- [18] V. Cortier, D. Galindo, R. Küsters, J. Müller, and T. Truderung, "SoK: Verifiability Notions for E-Voting Protocols," in *IEEE 37th Symposium on Security and Privacy (S&P 2016)*. IEEE Computer Society, 2016, pp. 779–798.
- [19] B. Smyth, "Ballot secrecy with malicious bulletin boards," Cryptology ePrint Archive, Tech. Rep. 2014/822, 2014.
- [20] V. Cortier and J. Lallemand, "Voting: You Can't Have Privacy without Individual Verifiability," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*. ACM, 2018, pp. 53–66.
- [21] B. Adida, "Helios: Web-based Open-Audit Voting," in *Proceedings of the 17th USENIX Security Symposium*, P. C. van Oorschot, Ed. USENIX Association, 2008, pp. 335–348.
- [22] V. Cortier, P. Gaudry, and S. Glondu, "Belenios: A Simple Private and Verifiable Electronic Voting System," in *Foundations of Security, Protocols, and Equational Reasoning - Essays Dedicated to Catherine A. Meadows*, ser. Lecture Notes in Computer Science, vol. 11565. Springer, 2019, pp. 214–238.
- [23] M. R. Clarkson, S. Chong, and A. C. Myers, "Civitas: Toward a Secure Voting System," in *2008 IEEE Symposium on Security and Privacy (S&P 2008)*. IEEE Computer Society, 2008, pp. 354–368.
- [24] C. Culnane and S. A. Schneider, "A Peered Bulletin Board for Robust Use in Verifiable Voting Systems," in *IEEE 27th Computer Security Foundations Symposium, CSF 2014, Vienna, Austria, 19-22 July, 2014*. IEEE Computer Society, 2014, pp. 169–183.
- [25] J. Feigenbaum, A. D. Jaggard, and R. N. Wright, "Open vs. Closed Systems for Accountability," in *Proceedings of the 2014 Symposium and Bootcamp on the Science of Security, HotSoS 2014, Raleigh, NC, USA, April 08 - 09, 2014*. ACM, 2014, p. 4.
- [26] R. Küsters, T. Truderung, and A. Vogt, "Accountability: Definition and Relationship to Verifiability," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS 2010)*. ACM, 2010, pp. 526–535, the full version is available at <http://eprint.iacr.org/2010/236>.
- [27] M. Graf, R. Küsters, and D. Rausch, "AUC: Accountable Universal Composability," in *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2023, pp. 1148–1167.
- [28] R. Künnemann, I. Esiyok, and M. Backes, "Automated Verification of Accountability in Security Protocols," in *32nd IEEE Computer Security Foundations Symposium, CSF 2019, Hoboken, NJ, USA, June 25-28, 2019*. IEEE, 2019, pp. 397–413.
- [29] N. Asokan, V. Shoup, and M. Waidner, "Asynchronous Protocols for Optimistic Fair Exchange," in *Proceedings of the IEEE Symposium on Research in Security and Privacy*. IEEE Computer Society, 1998, pp. 86–99.
- [30] C. Chang and Y. Chang, "Efficient Anonymous Auction Protocols with Freewheeling Bids," *Comput. Secur.*, vol. 22, no. 8, pp. 728–734, 2003.
- [31] Z. Zhao, M. Naseri, and Y. Zheng, "Secure Quantum Sealed-bid Auction with Post-confirmation," *Optics Communications*, vol. 283, no. 16, pp. 3194–3197, 2010.
- [32] C. Hong, J. Katz, V. Kolesnikov, W. Lu, and X. Wang, "Covert Security with Public Verifiability: Faster, Leaner, and Simpler," in *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part III*, ser. Lecture Notes in Computer Science, vol. 11478. Springer, 2019, pp. 97–121.
- [33] Y. Aumann and Y. Lindell, "Security Against Covert Adversaries: Efficient Protocols for Realistic Adversaries," in *Proceedings of the 4th Theory of Cryptography Conference (TCC 2007)*, ser. Lecture Notes in Computer Science, S. P. Vadhan, Ed., vol. 4392. Springer, 2007, pp. 137–156.
- [34] G. Asharov and C. Orlandi, "Calling Out Cheaters: Covert Security with Public Verifiability," in *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Beijing, China, December 2-6, 2012. Proceedings*, ser. Lecture Notes in Computer Science, vol. 7658. Springer, 2012, pp. 681–698.
- [35] C. Baum, I. Damgård, and C. Orlandi, "Publicly Auditable Secure Multi-Party Computation," in *Security and Cryptography for Networks - 9th International Conference, SCN 2014, Amalfi, Italy, September 3-5, 2014. Proceedings*, ser. Lecture Notes in Computer Science, vol. 8642. Springer, 2014, pp. 175–196.
- [36] M. Rivinius, P. Reisert, D. Rausch, and R. Küsters, "Publicly Accountable Robust Multi-Party Computation," in *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 2022, pp. 2430–2449.
- [37] M. Rivinius, "MPC with publicly identifiable abort from pseudorandomness and homomorphic encryption," in *Eurocrypt 2025, 2025*, (To appear).
- [38] H. Leibowitz, A. Herzberg, and E. Syta, "Provable Security for PKI Schemes," Cryptology ePrint Archive, Tech. Rep. 2019/807, 2019.
- [39] S. Matsumoto and R. M. Reischuk, "Certificates-as-an-Insurance: Incentivizing accountability in SSL/TLS," in *Proceedings of the NDSS Workshop on Security of Emerging Network Technologies (SENT'15)*, 2015.
- [40] T. H. Kim, L. Huang, A. Perrig, C. Jackson, and V. D. Gligor, "Accountable Key Infrastructure (AKI): a Proposal for a Public-key Validation Infrastructure," in *22nd International World Wide Web Conference, WWW '13, Rio de Janeiro, Brazil, May 13-17, 2013*. International World Wide Web Conferences Steering Committee / ACM, 2013, pp. 679–690.
- [41] M. Graf, R. Küsters, and D. Rausch, "Accountability in a Permissioned Blockchain: Formal Analysis of Hyperledger Fabric," in *IEEE European Symposium on Security and Privacy, EuroS&P 2020, Genoa, Italy, September 7-11, 2020*. Los Alamitos, CA, USA: IEEE, 2020, pp. 236–255.
- [42] G. O. Karame, E. Androulaki, M. Roeschlin, A. Gervais, and S. Capkun, "Misbehavior in Bitcoin: A Study of Double-Spending and Accountability," *ACM Trans. Inf. Syst. Secur.*, vol. 18, no. 1, pp. 2:1–2:32, 2015.
- [43] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," *CoRR*, vol. abs/1710.09437, 2017.
- [44] R3, "R3 Corda Master Documentation," <https://docs.corda.net/docs/corda-os/4.4.html>, 2020, (Accessed on 04/24/2020).
- [45] Ethereum Foundation, "Ethereum Enterprise," <https://www.ethereum.org/enterprise/>, 2019, (Accessed on 11/13/2019).
- [46] A. Boudguiga, N. Bouzerna, L. Granboulan, A. Olivereau, F. Quesnel, A. Roger, and R. Sirdey, "Towards Better Availability and Accountability for IoT Updates by Means of a Blockchain," in *2017 IEEE European Symposium on Security and Privacy Workshops, EuroS&P Workshops 2017, Paris, France, April 26-28, 2017*. IEEE, 2017, pp. 50–58.
- [47] G. D'Angelo, S. Ferretti, and M. Marzolla, "A Blockchain-based Flight Data Recorder for Cloud Accountability," in *Proceedings of the 1st Workshop on Cryptocurrencies and Blockchains for Distributed Systems, CRYBLOCK@MobiSys 2018, Munich, Germany, June 15, 2018*. ACM, 2018, pp. 93–98.
- [48] E. Funk, J. Riddell, F. Ankel, and D. Cabrera, "Blockchain technology: a data framework to improve validity, trust, and accountability of information exchange in health professions education," *Academic Medicine*, vol. 93, no. 12, pp. 1791–1794, 2018.

- [49] A. Shamis, P. Pietzuch, B. Canakci, M. Castro, C. Fournet, E. Ashton, A. Chamayou, S. Clebsch, A. Delignat-Lavaud, M. Kerner *et al.*, “IA-CCF: Individual Accountability for Permissioned Ledgers,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 467–491.
- [50] R. Küsters, J. Liedtke, J. Müller, D. Rausch, and A. Vogt, “Ordinos: A Verifiable Tally-Hiding E-Voting System,” in *IEEE European Symposium on Security and Privacy, EuroS&P 2020, Genoa, Italy, September 7-11, 2020*. IEEE, 2020, pp. 216–235.
- [51] C. C. Dragan, F. Dupressoir, K. Gjøsteen, T. Haines, P. B. Rønne, and M. R. Solberg, “Machine-Checked Proofs of Accountability: How to sElect Who is to Blame,” in *Computer Security - ESORICS 2023 - 28th European Symposium on Research in Computer Security, The Hague, The Netherlands, September 25-29, 2023, Proceedings, Part III*, ser. Lecture Notes in Computer Science, vol. 14346. Springer, 2023, pp. 471–491.
- [52] J. Heather and D. Lundin, “The Append-Only Web Bulletin Board,” in *Formal Aspects in Security and Trust, 5th International Workshop, FAST 2008, Malaga, Spain, October 9-10, 2008, Revised Selected Papers*, ser. Lecture Notes in Computer Science, vol. 5491. Springer, 2008, pp. 242–256.
- [53] V. Cortier and B. Smyth, “Attacking and fixing Helios: An analysis of ballot secrecy,” *Journal of Computer Security*, vol. 21, no. 1, pp. 89–148, 2013.
- [54] R. Küsters, T. Truderung, and A. Vogt, “Verifiability, Privacy, and Coercion-Resistance: New Insights from a Case Study,” in *32nd IEEE Symposium on Security and Privacy (S&P 2011)*. IEEE Computer Society, 2011, pp. 538–553.
- [55] R. Küsters, J. Müller, E. Scapin, and T. Truderung, “sElect: A Lightweight Verifiable Remote Voting System,” in *IEEE 29th Computer Security Foundations Symposium (CSF 2016)*. IEEE Computer Society, 2016, pp. 341–354.
- [56] N. Huber, R. Küsters, T. Krips, J. Liedtke, J. Müller, D. Rausch, P. Reisert, and A. Vogt, “Kryvos: Publicly tally-hiding verifiable e-voting,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM, 2022, pp. 1443–1457. [Online]. Available: <https://doi.org/10.1145/3548606.3560701>
- [57] R. Küsters, T. Truderung, and A. Vogt, “Formal Analysis of Chaumian Mix Nets with Randomized Partial Checking,” in *35th IEEE Symposium on Security and Privacy (S&P 2014)*. IEEE Computer Society, 2014, pp. 343–358.
- [58] R. Küsters and T. Truderung, “Security Analysis of Re-Encryption RPC Mix Nets,” in *IEEE 1st European Symposium on Security and Privacy (EuroS&P 2016)*. IEEE Computer Society, 2016, pp. 227–242.
- [59] P. Chaidos, V. Cortier, G. Fuchsbaue, and D. Galindo, “BeleniosRF: A Non-interactive Receipt-Free Electronic Voting Scheme,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. ACM, 2016, pp. 1614–1625.
- [60] S. Delaune, S. Kremer, and M. Ryan, “Coercion-Resistance and Receipt-Freeness in Electronic Voting,” in *Proceedings of the 19th IEEE Computer Security Foundations Workshop (CSFW’06)*. IEEE Computer Society Press, 2006, pp. 28–39.
- [61] R. Canetti, “Universally Composable Security,” *J. ACM*, vol. 67, no. 5, pp. 28:1–28:94, 2020.
- [62] R. Küsters, M. Tuengerthal, and D. Rausch, “The IITM Model: a Simple and Expressive Model for Universal Composability,” *Journal of Cryptology*, vol. 33, no. 4, pp. 1461–1584, 2020.
- [63] J. Camenisch, S. Krenn, R. Küsters, and D. Rausch, “iUC: Flexible Universal Composability Made Simple,” in *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III*, ser. Lecture Notes in Computer Science, vol. 11923. Springer, 2019, pp. 191–221, the full version is available at <http://eprint.iacr.org/2019/1073>.
- [64] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. D. Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolic, S. W. Cocco, and J. Yellick, “Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains,” in *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23-26, 2018*. ACM, 2018, pp. 30:1–30:15.
- [65] S. Kremer and P. B. Rønne, “To Du or Not to Du: A Security Analysis of Du-Vote,” in *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*. IEEE, 2016, pp. 473–486.
- [66] V. Cortier and B. Smyth, “Attacking and Fixing Helios: An Analysis of Ballot Secrecy,” in *Proceedings of the 24th IEEE Computer Security Foundations Symposium, CSF, 2011, 2011*, pp. 297–311.
- [67] D. Mestel, J. Müller, and P. Reisert, “How efficient are replay attacks against vote privacy? A formal quantitative analysis,” *J. Comput. Secur.*, vol. 31, no. 5, pp. 421–467, 2023.
- [68] J. Groth, “Simulation-Sound NIZK Proofs for a Practical Language and Constant Size Group Signatures,” in *Advances in Cryptology - ASIACRYPT 2006, 12th International Conference on the Theory and Application of Cryptology and Information Security, Shanghai, China, December 3-7, 2006, Proceedings*, ser. Lecture Notes in Computer Science, vol. 4284. Springer, 2006, pp. 444–459.

APPENDIX A
SUPPLEMENTARY FIGURES

Here we provide supplementary Figures 1 to 3.

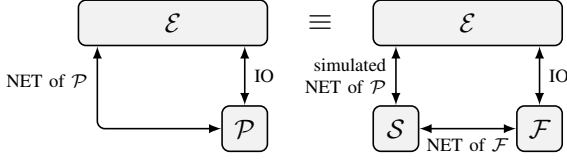
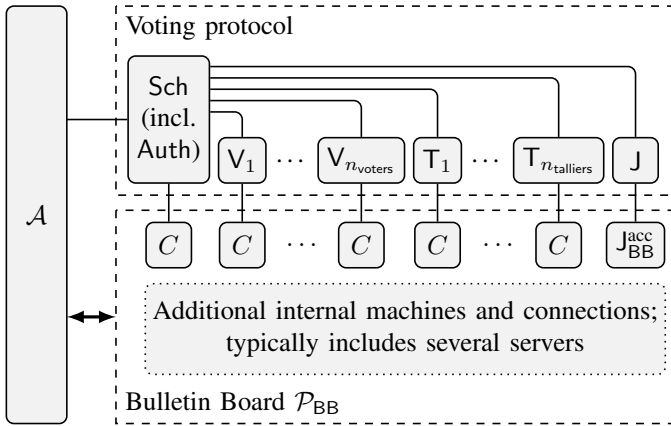


Fig. 1. UC security definition. Real world/protocol on the left, ideal world/protocol on the right. “IO” denotes input/output interfaces provided to higher-level protocols, “NET” are interfaces for sending/receiving network messages, $\equiv$ is computational indistinguishability of the output of $\mathcal{E}$.

$\mathcal{P}_{\text{ABOVE}}$:



Hybrid system $\pi_{\text{ideal-BB}}$:

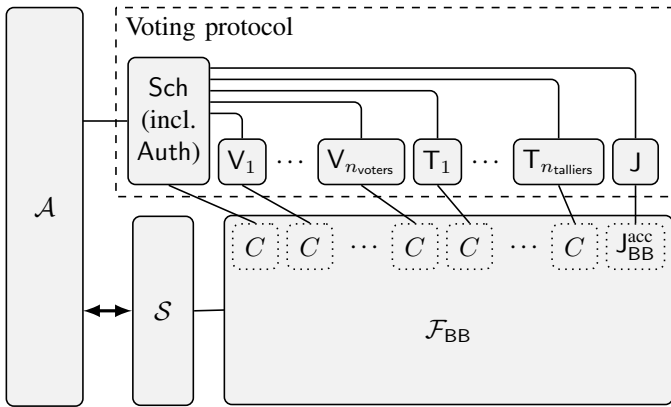


Fig. 2. Top: Illustration of the computational model of our protocol $\mathcal{P}_{\text{ABOVE}}$ (see Section V-A) running with an adversary $\mathcal{A}$. Bottom: Hybrid system used in the proof of Theorem 2 where the real BB $\mathcal{P}_{\text{BB}}$ is replaced by the ideal BB $\mathcal{F}_{\text{BB}}$ running with a simulator $\mathcal{S}$.

Lines indicate connected tapes that can be used for communication. The adversary can access and control the voting protocol machines via the scheduler Sch . The adversary also has direct connections to some or all of the machines in $\mathcal{P}_{\text{BB}}$ that are used for sending/receiving network messages; the double headed arrow indicates the set of all of these network connections, which depend on the specific protocol $\mathcal{P}_{\text{BB}}$.

$\mathcal{P}_{\text{ideal}}(n_{\text{voters}}, n_{\text{voters}}^{\text{hon}}, \mathcal{C}, \mu_{\text{vote}})[\mathbf{v}_b]$

Parameters:

- Probability distribution μ_{vote} over $\mathcal{C}$
- Numbers of voters n_{voters} and honest voters $n_{\text{voters}}^{\text{hon}}$
- $I \leftarrow \emptyset$ (initially)

Inputs:

- Choice $\mathbf{v}_b$ for the voter under observation $\mathbf{v}_{\text{obs}}$.

$\mathcal{P}_{\text{ideal}}$ uses a scheduler Sch which forwards messages to and from $\mathcal{A}_{\text{ideal}}$ while taking care that the following messages are sent sequentially in the order presented here. It also ensures that the first and last message ((init, honest) and compute) are sent at most once each:

On (init, honest) from $\mathcal{A}_{\text{ideal}}$ do:

- 1) $\forall i \in \{1, \dots, n_{\text{voters}}^{\text{hon}} - 1\}$: store $\mathbf{v}_i \xleftarrow{\mu_{\text{vote}}} \mathcal{C}$
- 2) Store $\mathbf{v}_{n_{\text{voters}}^{\text{hon}}} \leftarrow \mathbf{v}_b$
- 3) $I \leftarrow I \cup \{1, \dots, n_{\text{voters}}^{\text{hon}}\}$
- 4) Return success (to the adversary $\mathcal{A}_{\text{ideal}}$).

On (setChoice, $i, \mathbf{v}$) from $\mathcal{A}_{\text{ideal}}$ do:

- 1) If $i \notin \{n_{\text{voters}}^{\text{hon}} + 1, \dots, n_{\text{voters}}\}$ or $\mathbf{v} \notin \mathcal{C}$, return $\perp$.
- 2) Store $\mathbf{v}_i \leftarrow \mathbf{v}$
- 3) $I \leftarrow I \cup \{i\}$
- 4) Return success (to the adversary $\mathcal{A}_{\text{ideal}}$).

On compute from $\mathcal{A}_{\text{ideal}}$ do:

- 1) Return $\text{res} \leftarrow \left(\sum_{i \in [n_{\text{voters}}], \mathbf{v}_i = j} 1 \right)_{j \in [n_{\text{cand}}]}$ (to the adversary $\mathcal{A}_{\text{ideal}}$).

Fig. 3. The ideal voting protocol $\mathcal{P}_{\text{ideal}}$ for choice space $\mathcal{C}$, number n_{voters} of voters, $n_{\text{voters}}^{\text{hon}} \leq n_{\text{voters}}$ of which are honest voters, and vote distribution μ_{vote} . It formalizes that, in a perfectly secure voting system, an adversary does not learn anything about the vote $\mathbf{v}_b$ of a specific voter $\mathbf{v}_{\text{obs}}$ except for what the public election result reveals. The result is guaranteed to be an aggregation containing (at least) $n_{\text{voters}}^{\text{hon}}$ many honest votes which hide the individual vote $\mathbf{v}_b$, i.e., the adversary cannot break result computation to leak $\mathbf{v}_b$ directly.

APPENDIX B
FURTHER INSIGHTS RELATED TO PRIVACY

Equivocation Attacks for Threshold Encryption. Consider a homomorphic aggregation-based voting system that uses a threshold $t < n_{\text{talliers}}$ for decryption (unlike ABOVE, which uses full threshold, i.e., $t = n_{\text{talliers}}$). This is a common technique to improve the robustness of the tallying phase. In such systems, a malicious BB can run the following type of attack using equivocation.

If $t \leq \frac{n_{\text{talliers}}}{2}$, the malicious BB controlled by an adversary $\mathcal{A}$ can split the talliers into two or more distinct groups. It can show each group a different set of ballots, say, one set that contains all submitted ballots and one set that contains all ballots except for the one from $\mathbf{v}_{\text{obs}}$ that $\mathcal{A}$ wants to learn. As both sets of ballots appear reasonable to the respective

talliers, both groups would compute and publish the election result based on the respective sets of ballots they see. Hence, the adversary controlling the BB learns (at least) two election results (rather than only one as in an ideal voting protocol) and can even derive the vote of v_{obs} directly from both results.

This type of attack is not limited to $t \leq \frac{n_{\text{talliers}}}{2}$ but works for any threshold $t < n_{\text{talliers}}$ if the adversary, in addition to controlling the BB, also controls (up to) $t-1$ malicious talliers. As a result, even if $t = n_{\text{talliers}} - 1$ (such that there are $t-1 = n_{\text{talliers}} - 2$ malicious and 2 honest talliers), the BB might show each honest tallier a different set of ballots to learn one partial decryption for each of them. The malicious talliers can then compute the remaining partial decryptions for both sets.

We note that this is not an impossibility result. It rather demonstrates that, if e-voting protocols using a threshold $t < n_{\text{talliers}}$ are designed based on a real BB that can equivocate, then the protocol has to consider and protect against additional types of attacks on privacy.

A Gap in the Privacy Proof in [11]. While analyzing the privacy of ABOVE and checking whether we can re-use some arguments from privacy proofs of similar homomorphic-aggregation-based systems, we found a gap in the privacy proof given for Helios in [11].

Specifically, as part of their proof in [11], the authors perform a reduction to (a variant of) the CPA-security of ElGamal encryption under the assumption that there is at least one honest tallier T_{hon} . Intuitively, in the reduction, T_{hon} computes its public key share based on the public key shares of all other talliers in a way that forces the overall public encryption key of the election to be the same as the public key in the CPA game. While this works if T_{hon} happens to be the last tallier to publish its share, this is not guaranteed. If a malicious tallier waits until all other talliers have published their public key shares and then chooses its share depending on those published shares, say, by hashing all of them and using the result as the secret exponent, the reduction fails.

Fortunately, we found that a different reduction technique from [3] given for the threshold ElGamal version used by Benenios can be adapted to the full threshold ElGamal version used by Helios and ABOVE to show privacy, as described in Appendix F. This reduction is more complicated than the one given in [11] but does not require T_{hon} to compute its share based on shares of other talliers. Hence, it works irrespective of the order in which talliers publish shares.

APPENDIX C

CRYPTOGRAPHIC PRIMITIVES AND ASSUMPTIONS

A. Digital Signature Scheme

Definition 4 (Digital Signature Scheme). A digital signature scheme consists of a triple of polytime algorithms (KeyGen, Sign, Verify), where

- 1) KeyGen, the key generation algorithm, is a probabilistic algorithm that takes a security parameter η and returns a pair (pk, sk) of matching secret signing and public verification keys.

- 2) Sign, the signing algorithm, is a probabilistic algorithm that takes a private signing key sk and a message $x \in \{0, 1\}^*$ to produce a signature σ .
- 3) Verify, the verification algorithm, is a deterministic algorithm which takes a public verification key pk , a message $x \in \{0, 1\}^*$ and a signature σ to produce a boolean value.

We require that for all key pairs (pk, sk) which can be output by $\text{KeyGen}(1^\eta)$, for all messages $x \in \{0, 1\}^*$, and for all signatures σ that can be output by $\text{Sign}(sk, x)$, we have that $\text{Verify}(pk, x, \sigma) = \text{true}$.

Definition 5 (EUF-CMA-security). Let $S = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a signature scheme with security parameter η . Then S is existentially unforgeable under adaptive chosen-message attacks (EUF-CMA-secure) if for every ppt algorithm $\mathcal{A}$ who has access to a signing oracle and who never outputs tuples (x, σ) for which x has previously been signed by the oracle, we have that

$$\begin{aligned} \Pr[(pk, sk) \leftarrow \text{KeyGen}(1^\eta); \\ (x, \sigma) \leftarrow \mathcal{A}^{\text{Sign}(sk, \cdot)}(1^\eta, pk); \\ \text{Verify}(pk, x, \sigma) = \text{true}] \end{aligned}$$

is negligible as a function in η .

B. Non-Interactive Zero Knowledge Proof of Knowledge

There are several different ways to define security of NIZKPs. Here we follow [11] who define multi-theorem adaptive security based on a random oracle (RO). In principle, our proofs can be adapted to other variants of NIZKP definitions, such as ones based on common reference strings (CRS), as long as they provide similar properties. Notably, in the context of e-voting we require adaptive security, i.e., security should hold true even if a malicious prover (e.g., a voter) can dynamically determine the public statement that is to be shown (such as an encrypted ballot).

Definition 6 (NIZKP). Let $R \subseteq \mathcal{P}(\{0, 1\}^* \times \{0, 1\}^*)$ an (efficiently decidable) NP relation with corresponding NP language $\mathcal{L}_R = \{x \in \{0, 1\}^* \mid \exists w \in \{0, 1\}^* : (x, w) \in R\}$. Let $\Lambda \supseteq \mathcal{L}_R$ be an efficiently decidable superset of $\mathcal{L}_R$ which determines, e.g., which statements are considered for proofs (since statements $x \notin \Lambda$ can be detected and discarded without requiring a proof).

A non-interactive proof system for language $\mathcal{L}_R$ is a pair of ppt algorithms (Prove, Verify), where

- Prove takes as input a statement $x \in \mathcal{L}_R$, and a witness w such that $(x, w) \in R$. It implicitly also takes a security parameter 1^η as input and has access to a random oracle $\mathcal{RO}$ which it can query with inputs y to obtain $\mathcal{RO}(y)$. It returns a proof $\pi \in \{0, 1\}^*$.
- Verify takes as input the security parameter η , a statement $x \in \Lambda$, and a potential proof π for that statement. It also has implicit input 1^η and access to a random oracle $\mathcal{RO}$. It is deterministic and returns a boolean value.

A proof system is complete for $\mathcal{L}_R$ if, for every $(x, w) \in R$, we have: if $\pi \leftarrow^{\$} \text{Prove}(x, w)$ then $\text{Verify}(x, \pi) = 1$.⁹

A non-interactive zero knowledge proof of knowledge (NIZKP) for a language $\mathcal{L}_R$ is a complete non-interactive proof system for $\mathcal{L}_R$ that is zero knowledge and simulation sound extractable as defined next.

Definition 7 (Zero Knowledge Simulator). A simulator $\mathcal{S}$ for a proof system is a ppt algorithm in charge of answering random oracle queries and producing valid proofs for statements $x \in \Lambda$. Notably, it can patch the oracle to (try to) obtain valid proofs even without knowing a witness and even for statements $x \in \Lambda \setminus \mathcal{L}_R$. Specifically, $\mathcal{S}$ responds to the following queries:

- $\mathcal{RO}(y)$: $\mathcal{S}$ maintains a list of a list of oracle query/response pairs. For repeated queries, $\mathcal{S}$ answers consistently; for fresh queries, $\mathcal{S}$ draws a random value r in the range of the random oracle $\mathcal{RO}$, adds (y, r) to its list and returns r .
- $\text{Simulate}(x)$: For $x \in \Lambda$ the simulator returns a proof π such that $\text{Verify}(x, \pi) = 1$ if the verifier uses the simulator for its random oracle. $\mathcal{S}$ can add query/response pairs to its oracle list to process a Simulate query.

Definition 8 (Zero Knowledge). A proof system is zero-knowledge if there is a ppt simulator $\mathcal{S}$ such that no ppt adversary who can make queries to the random oracle and queries of the form $\text{Create-Proof}(x, w)$ can distinguish the following two settings with non-negligibly better than $\frac{1}{2}$ probability:

- 1) Random oracle queries are answered by a random oracle. In response to $\text{Create-Proof}(x, w)$, the challenger $\mathcal{CH}$ checks that $(x, w) \in R$. If not, $\mathcal{CH}$ returns $\perp$. Otherwise, $\mathcal{CH}$ returns $\text{Prove}(x, w)$.
- 2) The challenger $\mathcal{CH}$ runs a copy of the simulator $\mathcal{S}$. It forwards random oracle queries to $\mathcal{S}$ and returns the response. For $\text{Create-Proof}(x, w)$, $\mathcal{CH}$ checks that $(x, w) \in R$. If not, $\mathcal{CH}$ returns $\perp$. Otherwise, $\mathcal{CH}$ sends $\text{Simulate}(x)$ to $\mathcal{S}$ and returns the result to the adversary.

For the following definition, we consider ppt adversaries $\mathcal{A}$ that expect oracle access to the two methods provided by a ZK simulator $\mathcal{S}$. Notably and unlike the adversary in the previous definition, $\mathcal{A}$ may call $\text{Simulate}(x)$ for $x \in \Lambda \setminus \mathcal{L}_R$. At the end, $\mathcal{A}$ outputs two vectors $(\vec{x}, \vec{\pi})$ of the same length, where $\vec{x}$ contains only statements $x_i \in \Lambda$ and π_i is a claimed proof for statement x_i .

Definition 9 (Simulation Sound Extractability). Let Π be a zero knowledge proof system with simulator $\mathcal{S}$. We say that Π is simulation sound extractable (SSE) if there exists a ppt extractor $\mathcal{K}$ such that for every ppt adversary $\mathcal{A}$ as described above, $\mathcal{K}$ wins the following game with overwhelming proba-

bility:¹⁰

- 1) Initial run: The game selects a random string ω for $\mathcal{A}$. It runs an instance of $\mathcal{A}$ with the simulator $\mathcal{S}$ until $\mathcal{A}$ outputs and halts. If $\mathcal{A}$ does not output any proofs, any of the proofs do not verify (w.r.t. the instance of $\mathcal{S}$ used as the random oracle) or any of $\mathcal{A}$'s statement/proof pairs (x_i, π_i) is such that π was the result of a $\text{Simulate}(x)$ query, then $\mathcal{K}$ wins the game directly.
- 2) Extraction: The game runs an instance of $\mathcal{K}$, giving it the transcript of all queries in the initial run and the output $(\vec{x}, \vec{\pi})$ as input. $\mathcal{K}$ may repeatedly make one type of query Invoke in response to which the game runs a new invocation of $\mathcal{A}$ on the same randomness ω that it chose for the initial run. All queries made by these instances are forwarded to $\mathcal{K}$ who can reply to them.
- 3) Winning: $\mathcal{K}$ wins the game if it can output a vector of witnesses $\vec{w}$ that match the statements $\vec{x}$, i.e., for all i we have $(x_i, w_i) \in R$.

Simulation sound extractability combines several other common NIZKP properties, such as proof of knowledge and (adaptive) soundness, into one property. It further requires that soundness cannot be broken even if an adversary has access to some simulated proofs. In [11] it was shown that NIZKPs with this property can be constructed using the standard Fiat-Shamir heuristic by including the statement x_i in the hash.

C. Decisional Diffie-Hellman (DDH) Assumption

Definition 10 (DDH Assumption). Let $\text{GroupGen}(1^\eta)$ be a ppt algorithm that outputs tuples $(\mathbb{G}, p, g)$, where $\mathbb{G}$ is a description of a cyclical group of size p and with generator g . Let $\mathcal{A}$ be a ppt algorithm. The DDH experiment $\text{Exp}^{\text{DDH}-b'}$, $b' \in \{0, 1\}$, for p and $\mathcal{A}$ is defined as follows:

```

function  $\text{Exp}^{\text{DDH}-b'}(1^\eta)$ 
   $(\mathbb{G}, p, g) \leftarrow^{\$} \text{GroupGen}(1^\eta)$ ;
   $a \leftarrow^{\$} \mathbb{Z}_p, b \leftarrow^{\$} \mathbb{Z}_p, c \leftarrow^{\$} \mathbb{Z}_p$ ;
  if  $b' = 1$  then
    return  $\mathcal{A}(1^\eta, (\mathbb{G}, p, g), (g^a, g^b, g^{ab}))$ 
  else
    return  $\mathcal{A}(1^\eta, (\mathbb{G}, p, g), (g^a, g^b, g^c))$ 
  end if
end function

```

The DDH assumption holds for GroupGen if for every polynomial time algorithm $\mathcal{A}$ it holds true that there exists a negligible function f such that:

$$\left| \Pr[\text{Exp}^{\text{DDH}-1}(1^\eta) = 1] - \Pr[\text{Exp}^{\text{DDH}-0}(1^\eta) = 1] \right| \leq f(1^\eta)$$

¹⁰The definition given in [11] actually only states “non-negligible” instead of “overwhelming” but this seems to be a simple mistake in the text. Firstly, the authors of [11] state that their definition of simulation sound extractability for RO-based NIZKPs is intended as an analogue of simulation sound extractability for CRS-based NIZKPs given in [68] which requires a negligible probability for failing extraction. This is equivalent to succeeding with overwhelming probability. Secondly, the proofs of Theorems 2 and 3 in [11] use that the knowledge extractor (almost) always succeeds at computing witnesses, i.e., has an overwhelming winning probability.

⁹This can be relaxed to hold only with overwhelming probability. All of our results still hold in this case.

D. IND-CPA Secure Asymmetric Encryption

Definition 11 (IND-CPA security). Let $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be an asymmetric encryption scheme with security parameter η . Then $\mathcal{E}$ is indistinguishable under chosen-plaintext attacks (IND-CPA-secure) if for every pair of ppt algorithms $\mathcal{A} = (\mathcal{AF}, \mathcal{AG})$, we have that

$$\begin{aligned} & \Pr[(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\eta); \\ & \quad (x_0, x_1) \leftarrow \mathcal{AF}(1^\eta, \text{pk}); \\ & \quad b \leftarrow^{\$} \{0, 1\} \\ & \quad \text{ct}_b \leftarrow \text{Enc}^{\text{pk}}(x_b) \\ & \quad b' \leftarrow \mathcal{AG}(1^\eta, \text{pk}, \text{ct}_b) \\ & \quad b = b'] \end{aligned}$$

is negligible as a function in η .

APPENDIX D

UC SECURE COMPOSED PROTOCOLS AND TRACE PROPERTIES

In this section we recall why UC security of a (possibly composed) protocol does not imply that arbitrary trace properties carry over between real and ideal world. We also give two simple counter examples. While there exist trace properties that indeed *do carry over for composed protocols* based on UC-security of an underlying real/ideal subroutine, it is therefore necessary to prove this result for a given trace property on a case by case basis. This is what we did in our verifiability proof, where we showed that our verifiability notion, which is defined as a trace property, applied to a composed e-voting protocol carries over based on UC-security of the underlying real/ideal BB (see Theorem 2 and Appendix E).

High-level picture. Recall that showing UC security of a real protocol $\mathcal{P}$ entails defining an ideal protocol $\mathcal{F}$ and a simulator $\mathcal{S}$ such that no ppt environment $\mathcal{E}$ which interacts either with $\mathcal{P}$ (real world) or $\mathcal{F}$ running with the simulator $\mathcal{S}$ (ideal world; written $\mathcal{F} \mid \mathcal{S}$) can distinguish both cases. In the ideal world, $\mathcal{F}$ handles inputs/outputs from/to $\mathcal{E}$ acting as an arbitrary higher-level protocol while $\mathcal{S}$ is responsible for simulating the network behavior of $\mathcal{P}$ towards $\mathcal{E}$ (cf. Figure 1).

The UC composition theorem then implies that a higher-level protocol $\mathcal{Q}$ using $\mathcal{P}$ as a subroutine (written $\mathcal{Q} \mid \mathcal{P}$) UC-realizes $\mathcal{Q}$ using $\mathcal{F}$ (written $\mathcal{Q} \mid \mathcal{F}$). More specifically, the composition theorem states that $\mathcal{Q} \mid \mathcal{P}$ and $\mathcal{Q} \mid \mathcal{F} \mid \mathcal{S}$ (for essentially the same simulator $\mathcal{S}$ as before) are indistinguishable for any ppt environment $\mathcal{E}'$. Intuitively, this holds because an environment $\mathcal{E}$ trying to distinguish $\mathcal{P}$ and $\mathcal{F} \mid \mathcal{S}$ can internally simulate $\mathcal{E}'$ and the higher protocol $\mathcal{Q}$.

One might hope that this composition result also implies that trace properties, i.e., events defined over runs of a systems, that were shown to hold true for $\mathcal{E}' \mid \mathcal{Q} \mid \mathcal{P}$ with a certain probability must hold with the same probability for the system $\mathcal{E}' \mid \mathcal{Q} \mid \mathcal{F} \mid \mathcal{S}$ (except for a negligible difference). This, however, is not the case: $\mathcal{P}$ UC-realizing $\mathcal{F}$ only implies that any externally observable behavior is

computationally indistinguishable, which is sufficient for UC-security of composed protocols. But a trace property might also depend on some internal behavior that is not observable for a UC environment. The probability of such a property can be drastically different for $\mathcal{P}$ and $\mathcal{F} \mid \mathcal{S}$, which have different internals. This will generally still be the case after composition with a higher-level protocol $\mathcal{Q}$. Even if a trace property is defined only over externally observable behavior, if the property itself cannot be decided within polynomial time, it might still differ between both systems by an arbitrarily large probability despite computational indistinguishability.

Counter example 1. Consider the protocol $\mathcal{P}$ which expects some input m from higher-level protocols/the environment, then sets an internal variable i to 0, and returns m as output. $\mathcal{P}$ ignores any messages received on the network. Define the protocol $\mathcal{F}$ to be the same as $\mathcal{P}$ but it sets i to 1. One easily verifies that $\mathcal{P}$ UC-realizes $\mathcal{F}$: take the simulator $\mathcal{S}$ that ignores incoming network messages. The input/output behavior of $\mathcal{P}/\mathcal{F}$ observable by a UC-environment are identical by definition and therefore indistinguishable.

Now consider the trace property defined via the event E which contains all runs where the internal variable i is set to 1. Note that this trace property cannot be computed by environments as it is defined over internal behavior which is not observable. For every environment $\mathcal{E}$ we have that $\Pr[E] = 0$ in the system $\mathcal{E} \mid \mathcal{P}$. However, for any environment $\mathcal{E}$ that sends at least one input we have $\Pr[E] = 1$ in the system $\mathcal{E} \mid \mathcal{F} \mid \mathcal{S}$. It is easy to see that the probabilities are the same for composed systems $\mathcal{E}' \mid \mathcal{Q} \mid \mathcal{P}$ and $\mathcal{E}' \mid \mathcal{Q} \mid \mathcal{F} \mid \mathcal{S}$ for every environment $\mathcal{E}'$ and higher-level protocol $\mathcal{Q}$ that are such that at least one input is sent to the subroutine $\mathcal{P}/\mathcal{F}$.

Of course, the above example is very artificial as it is only meant to illustrate the underlying problem. In the security analysis of an actual protocol from practice, such dependencies of trace properties on some internal unobservable behavior can be far less obvious.

Counter example 2. Let $f(\cdot, k)$ be a secure pseudo-random function where k is the secret key obtained via some key generation algorithm. Consider the protocol $\mathcal{P}$ which, upon first activation, runs the key generation algorithm to obtain and store k . $\mathcal{P}$ then responds to any input m from a higher-level protocols/the environment by returning output $f(m, k)$. $\mathcal{P}$ ignores any network messages. Define $\mathcal{F}$ to be a random function: when it receives an input m that it has never seen before, it returns a uniformly random output. When it receives an input m that was sent before, it returns the same output as it did previously. $\mathcal{F}$ ignores network messages.

We have that $\mathcal{P}$ UC-realizes $\mathcal{F}$ using the simulator $\mathcal{S}$ that ignores all network messages: If there were a ppt environment $\mathcal{E}$ that can distinguish both protocols, then we could use that same environment to distinguish $f(\cdot, k)$ from a random function, contradicting the assumption that f is a secure pseudo-random function.

Now consider the trace property defined via the following event E : Given the sequence of input-output pairs (m, out)

sent to/received from $\mathcal{P}/\mathcal{F}$ in a run r , E contains this run r iff there exists a key k such that $f(m, k) = \text{out}$ for all of these pairs. Note that, unlike for the previous counter example, E is defined only over externally observable behavior but cannot be computed by a polytime environment. For every environment $\mathcal{E}$ we have that $\Pr[E] = 1$ in the system $\mathcal{E} \mid \mathcal{P}$. However, in the system $\mathcal{E} \mid \mathcal{F} \mid \mathcal{S}$ the probability $\Pr[E]$ will be close to 0 as there are only very few sequences of random outputs that happen to correspond to a key; the exact probability depends on the size of the key set and the number of different inputs sent by $\mathcal{E}$. Just as for the previous counter example, the same gap will still exist after composition with a higher-level protocol $\mathcal{Q}$.

Again, the trace property E is a somewhat artificial example that is intended only to illustrate the issue. However, security definitions via trace properties used for the analysis of real world protocols can become very complex and typically do not explicitly require the definition itself to be decidable in polynomial time. It is then unclear whether such a given trace property carries over between real and ideal systems.

APPENDIX E PROOF OF E2E VERIFIABILITY

Here, we provide the full proof of Theorem 2.

Proof. Let $\mathcal{A}$ be an arbitrary ppt adversary and let $\pi_{\text{real-BB}} = (\mathcal{P}_{\text{ABOVE}} \mid \mathcal{A})$. Let $\mathcal{S}$ be the ppt UC simulator that exists due to $\mathcal{P}_{\text{BB}} \leq \mathcal{F}_{\text{BB}}$ (see Section II). Let $\pi_{\text{ideal-BB}}$ be the same as $\pi_{\text{real-BB}}$ but $\mathcal{P}_{\text{BB}}$ (in $\mathcal{P}_{\text{ABOVE}}$) is replaced by the combination of $\mathcal{F}_{\text{BB}}$ and $\mathcal{S}$, where $\mathcal{S}$ controls the network communication between $\mathcal{F}_{\text{BB}}$ and $\mathcal{A}$ just as it does in the UC security definition, cf. Figure 2. We need to show Definition 1 for $\gamma_k^{\text{gen}}(\phi)$ and $\delta_k(p_{\text{verif}})$, i.e., that $\Pr[\pi_{\text{real-BB}}(1^\eta) \mapsto \neg\gamma_k^{\text{gen}}(\phi), (J : \text{accept})]$ is $\delta_k(p_{\text{verif}})$ -bounded.

We show this result in two steps: First, we argue that this property holds for the idealized $\pi_{\text{ideal-BB}}$ instead of the real $\pi_{\text{real-BB}}$. Since $\pi_{\text{ideal-BB}}$ uses an ideal BB functionality, we can directly use accountability w.r.t. consistency in the proof and do not have to worry about any implementation details of the BB. In the second step, we show that verifiability for $\pi_{\text{real-BB}}$ follows by verifiability of $\pi_{\text{ideal-BB}}$. While at first glance, one might expect that this follows trivially from the UC composition theorem – after all, the UC composition theorem implies that $\pi_{\text{real-BB}}$ and $\pi_{\text{ideal-BB}}$ are indistinguishable for any ppt distinguishing environment – this is not actually the case. In general, UC indistinguishability does not imply that trace properties which are defined over runs of a system, such as our verifiability notion, also remain indistinguishable between real and ideal system. Intuitively, this is because trace properties might not be computable for UC environments, e.g., because the trace property can be defined over and depend on some internal variable that the UC environment cannot access. We discuss this gap between UC security and trace properties in depth for interested readers in Appendix D, including two simple counterexamples. Fortunately, we are able to show that our verifiability notion is defined in such a way that one can, in

fact, reduce verifiability of $\pi_{\text{real-BB}}$ to UC indistinguishability of $\mathcal{P}_{\text{BB}}$ and $\mathcal{F}_{\text{BB}}$ and verifiability of $\pi_{\text{ideal-BB}}$.

Step 1. We define the following events consisting of sets of runs of $\pi_{\text{ideal-BB}}(1^\eta)$:

- $\chi_{\text{ideal-BB}} := \neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$
- χ_1 is the subset of $\chi_{\text{ideal-BB}}$ where a signature for a public key belonging to an honest party is forged.
- χ_2 is the subset of $\chi_{\text{ideal-BB}} \setminus \chi_1$ where Step 1) of the judge J returns **reject**. In those runs, the accountability judging subroutine in $\mathcal{F}_{\text{BB}}$ has detected misbehavior such that there might have been inconsistent outputs to two (not necessarily distinct) honest parties.
- χ_3 is the subset of $\chi_{\text{ideal-BB}} \setminus \bigcup_{i=1, \dots, 2} \chi_i$ where Step 2) of the judge J returns **reject**, i.e., where the judge is unable to compute an election result.
- χ_4 is the subset of $\chi_{\text{ideal-BB}} \setminus \bigcup_{i=1, \dots, 3} \chi_i$ where Step 3) of the judge J returns **reject**, i.e., at least one voter issued a complaint with a valid signature.
- χ_5 is the subset of $\chi_{\text{ideal-BB}} \setminus \bigcup_{i=1, \dots, 4} \chi_i$ where a NIZKP π_{Enc} or π_{DecShare} (in an output of $\mathcal{F}_{\text{BB}}$ to an honest party) is valid but the statement is false, i.e., there is no witness.
- χ_6 is the subset of $\chi_{\text{ideal-BB}} \setminus \bigcup_{i=1, \dots, 5} \chi_i$ where two or more honest voters compute the same ciphertext vector **ct**.
- $\chi_7 := \chi_{\text{ideal-BB}} \setminus \bigcup_{i=1, \dots, 6} \chi_i$, i.e., the remaining runs in $\chi_{\text{ideal-BB}}$ that are not covered by a previous event.

Note that the χ_i are a partition of $\chi_{\text{ideal-BB}}$ and therefore we have $\Pr[\chi_{\text{ideal-BB}}] = \sum_{i=1, \dots, 7} \Pr[\chi_i]$. Also, note that $\neg\gamma_k^{\text{gen}}(\phi)$ implies that the honesty assumptions ϕ are met in all of these events. In particular, both the election authority Auth and the judge J follow the protocol, and the signing key of Auth is secret.

In what follows, we are going to show that $\Pr[\chi_1]$, $\Pr[\chi_5]$, and $\Pr[\chi_6]$ are negligible, $\Pr[\chi_2] = \Pr[\chi_3] = \Pr[\chi_4] = 0$, and $\Pr[\chi_7] \leq (1 - p_{\text{verif}})^{k+1} = \delta_k(p_{\text{verif}})$. Since $\Pr[\chi_{\text{ideal-BB}}] = \sum_{i=1, \dots, 7} \Pr[\chi_i]$, this implies that $\Pr[\chi_{\text{ideal-BB}}]$ is $\delta_k(p_{\text{verif}})$ -bounded and hence $\pi_{\text{ideal-BB}}$ is $(\gamma_k^{\text{gen}}(\phi), \delta_k(p_{\text{verif}}))$ -verifiable, concluding Step 1.

Event χ_1 . The probability of this event can be reduced to an adversary $\mathcal{A}'$ on the EUF-CMA-security of $\mathcal{S}$: The adversary $\mathcal{A}'$ first guesses one party p . It then simulates $\pi_{\text{ideal-BB}}(1^\eta)$ but replaces p 's public key with the challenge public key while using the challenge signing oracle to generate any signatures that p wants to compute. If party p is corrupted, then the simulation stops early. At the end of the run, $\mathcal{A}'$ checks whether a forgery has occurred for p 's public key and, if so, returns that forgery to win the EUF-CMA-security game.

Note that this simulation is perfect such that whenever a forgery occurs in $\pi_{\text{ideal-BB}}(1^\eta)$ and $\mathcal{A}'$ has previously guessed (one of the) parties p for whom a signature is forged in a run, $\mathcal{A}'$ wins the EUF-CMA-security game. We, therefore, have $\Pr[\chi_1] \leq \Pr[\mathcal{A}' \text{ wins the EUF-CMA game}] \cdot (\# \text{ parties})$. Since the number of parties is constant and $\mathcal{S}$ is EUF-CMA secure, this implies that $\Pr[\chi_1]$ is negligible.

Finally, observe that, by definition, in all of the following events $\chi_i, i \geq 2$ there are no forged signatures.

Events χ_2, χ_3, χ_4 . Since in these events the judge always returns `reject`, which falsifies $\chi_{\text{ideal-BB}}$, we trivially have $\Pr[\chi_i] = 0, i \in \{2, 3, 4\}$. The main purpose of these events is rather the following observations:

For all events $\chi_i, i \geq 3$, all honest parties who see parameters of the election – which include the set of eligible voters but also, e.g., the set of talliers and the choice space C (cf. The setup phase in Section III) – or who compute a (non-error) election result will see the same and hence agree on all parameters, respectively the result. Note that this does not yet guarantee that either actually exists, e.g., because there might not be any party who can compute an election result. For the election parameters, uniqueness follows because the trusted election authority Auth creates only a single signed message with all election parameters, and no forgery for its public key has occurred (cf. χ_1). Hence, if a party sees any election parameters with a valid signature, they must be the uniquely defined ones created by the election authority. For the election result, we know that the judge in the ideal $\mathcal{F}_{\text{BB}}$ has not been able to identify any misbehavior; otherwise, this run would have been in χ_2 in contradiction to the definition of χ_i . $\mathcal{F}_{\text{BB}}$ guarantees, in this case, that all clients belonging to honest parties always obtain consistent outputs, i.e., all outputs are a prefix of the same globally unique list of messages. Note that different parties might still see prefixes of different lengths, and hence, not all of them might be able to compute the election result, if it even exists. However, the election result computation is stable under extension in that, given the shortest sequence of messages returned by $\mathcal{F}_{\text{BB}}$ which allows for computing an election result, any extension of this sequence will give the same result. It follows that all parties who are able to compute an election result will obtain the same result, irrespective of how many messages $\mathcal{F}_{\text{BB}}$ did return to them.

For all events $\chi_i, i \geq 4$, we have that both election parameters (including the set of eligible voters) and a non-error election result exist because both were read/computed (at least) by the party running the judge J; otherwise, the run would be in χ_3 in contradiction to the definition of χ_i . Let res denote the uniquely defined non-error election result in what follows.

Furthermore, for all events $\chi_i, i \geq 4$, we also have that all parties who see setup finish will compute the same unique non-error public encryption key for the election. Agreement/uniqueness follows from χ_2 and because the computation of the public encryption key is stable under extension. The computation does not result in an error since otherwise, the judge would be unable to compute an election result, i.e., this run would be in χ_3 . Hence, all honest voters who see setup finish are able to encrypt their intended vote, will use the correct key, and generate a valid NIZKP π^{Enc} , which is bound to the public key that was used during encryption. If the resulting encrypted ballot ends up being included in the

set of tallied ballots, it will, therefore, not get dropped due to being invalid during tallying.

For all events $\chi_i, i \geq 5$, we have that all honest voters who did not want to abstain were able to compute an encrypted ballot that they then tried to submit via $\mathcal{F}_{\text{BB}}$. This follows because honest voters who do not see setup finished will always issue a complaint via a reliable channel, i.e., such a run would be in χ_4 . The only other case that might cause a failure to compute an encrypted ballot would occur if voters see the setup phase finish but then, due to wrong $\pi^{\text{KeyShareGen}}$ NIZKPs from at least one tallier, are unable to compute a public encryption key for the election. However, this cannot happen: By consistency of the output (cf. χ_2), if the setup phase finishes in such a way that the public encryption key cannot be computed, then the judge J who sees the same setup phase also cannot compute the public key. However, then the judge also cannot recompute a non-error election result and rejects, i.e., this run would be in χ_3 instead of χ_i .

Finally, for all events $\chi_i, i \geq 5$, we also have that vote verification succeeded for all honest voters who decided to perform it. This immediately follows from the definition of χ_4 and the assumption of reliable channels for sending complaints from voters to the judge J.

Event χ_5 . This event can be shown to be negligible via reductions to the security of π^{Enc} and π^{DecShare} (cf. Section C-B). More specifically, let β_{enc} be the event that, in a run of $\pi_{\text{ideal-BB}}(1^n)$, there is a valid NIZKP π^{Enc} in an output of $\mathcal{F}_{\text{BB}}$ to an honest party for which the statement is false, i.e., there does not exist a witness; analogously for β_{ds} and π^{DecShare} . Since $\chi_5 = (\beta_{\text{enc}} \cup \beta_{\text{ds}})$, for proving that χ_5 is negligible, it suffices to show that both of these events are negligible.

Consider β_{enc} ; the case of β_{ds} is analogous. Since π^{Enc} is assumed to be secure, there exists a ppt knowledge extractor $\mathcal{K}$ (and a ppt simulator $\mathcal{S}_{\text{zk}}$ offering a random oracle and a proof creation oracle) for the simulation sound extractability game that wins against every ppt adversary $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ creating NIZKPs with overwhelming probability. Now consider the following adversary: $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ simulates a run of $\pi_{\text{ideal-BB}}(1^n)$ where, whenever a random oracle query needs to be answered, $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ uses the random oracle provided by $\mathcal{S}_{\text{zk}}$ rather than running an actual random oracle as $\pi_{\text{ideal-BB}}(1^n)$ would. The adversary does not use the proof creation oracle provided by $\mathcal{S}_{\text{zk}}$. Once the simulated run has finished, $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ returns the list of all instances of π^{Enc} that were returned at some point by $\mathcal{F}_{\text{BB}}$ to any honest party and which are valid (according to the random oracle offered by $\mathcal{S}_{\text{zk}}$). One easily observes that $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ is indeed ppt and that the only difference compared to $\pi_{\text{ideal-BB}}(1^n)$ is due to how random oracle queries are handled.

Let $\gamma_{\mathcal{S}_{\text{zk}}}$ be the event that $\mathcal{K}$, in the simulation sound extractability game running with $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ and $\mathcal{S}_{\text{zk}}$, fails to output a witness for at least one NIZKP returned by $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$. This event implies that $\mathcal{K}$ loses its game as the output by $\mathcal{A}_{\text{sound}, \mathcal{S}_{\text{zk}}}$ passes the trivial win condition checks, such as whether all of its NIZKPs are valid. Since π^{Enc} is assumed to be secure, $\Pr[\gamma_{\mathcal{S}_{\text{zk}}}]$ is negligible.

Next, we must bridge the gap between $\mathcal{A}_{\text{sound}, S_{zk}}$ and $\pi_{\text{ideal-BB}}(1^\eta)$. Consider a variant of $\mathcal{A}_{\text{sound}, S_{zk}}$, called $\mathcal{A}_{\text{sound}, \mathcal{RO}}$ in the following, that does not use S_{zk} but rather uses an actual random oracle $\mathcal{RO}$. Note that $\mathcal{A}_{\text{sound}, \mathcal{RO}}$ perfectly simulates runs of $\pi_{\text{ideal-BB}}(1^\eta)$. Let $\gamma_{\mathcal{RO}}$ be the event that $\mathcal{K}$, in the simulation sound extractability game running with $\mathcal{A}_{\text{sound}, \mathcal{RO}}$, fails to output a witness for at least one NIZKP returned by $\mathcal{A}_{\text{sound}, \mathcal{RO}}$. We can reduce the difference $|\Pr[\gamma_{\mathcal{RO}}] - \Pr[\gamma_{S_{zk}}]|$ to the zero-knowledge property of β_{enc} , which intuitively states, among others, that a simulated random oracle must be indistinguishable from an actual one. More formally, define $\mathcal{D}_{zk}$ as the following ppt distinguisher for the zero-knowledge property: $\mathcal{D}_{zk}$ first runs $\mathcal{A}_{\text{sound}, \mathcal{RO}}$ resp. $\mathcal{A}_{\text{sound}, S_{zk}}$. Random oracle queries are answered by the random oracle provided by the zero-knowledge challenger, which is either an actual random oracle (hence yielding a run of $\mathcal{A}_{\text{sound}, \mathcal{RO}}$) or simulated by the simulator S_{zk} (hence yielding a run of $\mathcal{A}_{\text{sound}, S_{zk}}$). $\mathcal{D}_{zk}$ then runs $\mathcal{K}$ on the output, just as in the simulation sound extractability game. Finally, $\mathcal{D}_{zk}$ outputs 1 iff $\mathcal{K}$ fails to output a witness for at least one NIZKP returned by $\mathcal{A}_{\text{sound}, \mathcal{RO}}$ resp. $\mathcal{A}_{\text{sound}, S_{zk}}$. Note that this distinguisher is a ppt algorithm and outputs 1 iff the event $\Pr[\gamma_{\mathcal{RO}}]$ resp. $\Pr[\gamma_{S_{zk}}]$ occurs. By indistinguishability of the ZK property, we immediately get a negligible upper bound for $|\Pr[\gamma_{\mathcal{RO}}] - \Pr[\gamma_{S_{zk}}]|$.

Finally, we observe that $\Pr[\beta_{\text{enc}}] \leq \Pr[\gamma_{\mathcal{RO}}]$. This is because if there is no witness for a statement with a valid NIZKP in some run of $\pi_{\text{ideal-BB}}(1^\eta)$, then $\mathcal{K}$ will certainly be unable to output one. Altogether, we have $\Pr[\beta_{\text{enc}}] \leq \Pr[\gamma_{\mathcal{RO}}] \leq \Pr[\gamma_{S_{zk}}] + \text{negl} \leq \text{negl} + \text{negl}$. Repeating the same line of reasoning for β_{ds} then shows the initial claim that $\Pr[\chi_5]$ is negligible.

For all events $\chi_i, i \geq 6$, it, therefore, follows that a witness exists for all valid NIZKPs π^{Enc} and π^{DecShare} (returned to an honest party by $\mathcal{F}_{\text{BB}}$). We note that for the rest of the proof existence of witnesses is already sufficient, i.e., we do not need to extract the values of those witnesses.

Event χ_6 . We have that $\Pr[\chi_6]$ is negligible by Assumption (V2).¹¹

Event χ_7 . For the final event containing all remaining runs, we will show $\Pr[\chi_7] \leq \delta_k(p_{\text{verif}})$.

We start by observing that, by the definition of the previous events, in all runs of χ_7 , the judge J always accepts (cf. events χ_2 to χ_4), which (since $\chi_7 \subseteq \chi_{\text{ideal-BB}}$) implies that the goal $\gamma_k^{\text{gen}}(\phi)$ is not met in χ_7 . This can only be the case if the honesty assumptions ϕ hold true but at least one of the

¹¹Note that Assumption (V2) follows from the DDH assumption for non-trivial groups, i.e., groups with at least 2 elements: ElGamal ciphertexts include a fresh group element g^a chosen uniformly at random. Suppose by contradiction that after generating a fixed number of such group elements – one for each (honest) voter – there is a non-negligible chance for two such group elements to be identical. Then a ppt adversary on the DDH experiment who internally generates the same number of uniformly random exponents e_i would have a non-negligible chance that $g^{e_i} = g^a$, where g^a is a challenge group element. In non-trivial groups of size ≥ 2 , knowledge of the secret exponent $a = e_i$ allows for immediately winning the DDH game, thus contradicting the DDH assumption.

Conditions a), b), or c) in Definition 2 is not met. By events χ_3 and χ_4 , we have established that both the set of eligible voters and the election result exist and are unique, i.e., any honest party able to see/compute them will necessarily obtain the same set of voters/election result. Hence, Conditions a) and b) are met such that a run can only be in χ_7 because Condition c) is false, i.e., the distance between the actual election result res and the closest election result which can be obtained from the intended votes of all honest (eligible) voters is larger than k . Formally: Given a run $r \in \chi_7$, let $\mathbf{v}_1, \dots, \mathbf{v}_{n_{\text{voters}}^{\text{hon}}} \in \tilde{\mathcal{C}}$ be the intended votes by the $n_{\text{voters}}^{\text{hon}}$ honest voters $\text{id}_{\text{voters}}^{\text{hon}} \subseteq \text{id}_{\text{voters}}$ in r . Then, for all valid votes $\mathbf{v}'_1, \dots, \mathbf{v}'_{n_{\text{voters}}^{\text{dishon}}} \in \tilde{\mathcal{C}}$ that the $n_{\text{voters}}^{\text{dishon}}$ dishonest voters in r can choose and for all vectors of valid votes $\mathbf{vv}^{\text{real}} := (\mathbf{v}_i^{\text{real}} \in \tilde{\mathcal{C}})_{i \in \text{id}_{\text{voters}}}$ representing a vote distribution that yields the election result res from this run r , it follows that $\text{dist}(\mathbf{vv}^{\text{ideal}}, \mathbf{vv}^{\text{real}}) > k$ where $\mathbf{vv}^{\text{ideal}} := (\mathbf{v}_1, \dots, \mathbf{v}_{n_{\text{voters}}^{\text{hon}}}, \mathbf{v}'_1, \dots, \mathbf{v}'_{n_{\text{voters}}^{\text{dishon}}})$.

By this, we find that the votes contained in the ciphertexts that were aggregated and then decrypted to obtain the election result must, in particular, also have distance more than k from $\mathbf{vv}^{\text{ideal}}$. To see this, let $\mathbf{vv}^{\text{real}}$ to be the plain votes obtained by decrypting those encrypted ballots that are included in the aggregated ciphertext vector ct_{aggr} (note that each such encrypted ballot corresponds to one unique voter that signed it) and abstain for any voter who did not sign an encrypted ballot that was included in ct_{aggr} . Since only ballots with valid NIZKPs π^{Enc} are included in aggregation, witnesses for the statements corresponding to all those NIZKPs exist (cf. χ_5) and by correctness of Exponential ElGamal decryption (which yields a unique plaintext and therefore necessarily the witness plaintext), we have that these votes are well-defined and valid, i.e., $\mathbf{v}_i^{\text{real}} \in \tilde{\mathcal{C}}$ for all i . Since Exponential ElGamal is additively homomorphic and since the election result is computed only from tallier messages with valid NIZKPS π^{DecShare} (that have witnesses for their statement - cf. χ_5), we further have that summing all votes in $\mathbf{vv}^{\text{real}}$ yields the same election result res that is obtained by decrypting ct_{aggr} . Altogether, we conclude that this choice of votes $\mathbf{vv}^{\text{real}}$ has distance more than k to all possible correct vote distributions $\mathbf{vv}^{\text{ideal}}$; otherwise $\mathbf{vv}^{\text{real}}$ would contradict Condition c) being false.

Observe that $\mathcal{A}$ can vote for each malicious party at most once by the tallying procedure. Due to the unforgability of signatures (cf. χ_1), $\mathcal{A}$ cannot generate new votes for honest parties. $\mathcal{A}$, therefore, can neither add new votes in the name of abstaining voters nor modify encrypted ballots of honest voters. Since encrypted ballots of honest voters were generated with the correct public key (cf. χ_2), the intended vote contained therein will be counted correctly towards the election result if the encrypted ballot is included in the list of ballots that are aggregated while tallying. Therefore, the only option for an adversary to cause $\mathbf{vv}^{\text{real}}$ to deviate from a correct vote distribution $\mathbf{vv}^{\text{ideal}}$, which includes all intended votes $\mathbf{v}_1, \dots, \mathbf{v}_{n_{\text{voters}}^{\text{hon}}}$, is by dropping some of the (non-abstain) intended votes of honest voters.

By χ_4 , we know that all honest voters who did not intend to

abstain were able to generate and submit a signed, encrypted ballot containing their intended vote under the correct public encryption key. So there are only three ways left how such a vote can get dropped: (i) the submitted ballot is included in the BB state but at a position before setup has finished such that it will be ignored during tallying, (ii) the submitted ballot is included in the BB state at a position after setup has finished and before the first `VotingClosed` message – so it is considered during tallying – but is not aggregated and counted towards the election result due to ballot weeding, or (iii) the submitted ballot is not included on the BB before the first `VotingClosed` message. In all other cases, the correctness of the signature scheme, correctness of Exponential ElGamal encryption, and completeness of π^{Enc} imply the intended vote contained in the encrypted ballot will be counted towards the election result, i.e., is one of the votes in $\mathbf{vv}^{\text{real}}$.

Case (i): This case does not actually occur. By unforgability of signatures (cf. χ_1), only the honest voter can compute the signed ballot. Honest voters compute their ballots only after seeing the setup finished. Since all BB outputs are consistent with the same append-only state (cf. χ_2), those ballots can thus be added to the BB state only at a position after the setup is finished.

Case (ii): By event χ_6 , we know that the ballot weeding procedure did not drop any ballots due to two honest voters generating the same ciphertext vectors by accident. However, ballot weeding might drop the ballot of an honest voter because a malicious voter chooses to submit the same ciphertext vector under their signature. More specifically, an adversary controlling the network and possibly also the entire BB can do the following: When seeing a (signed) ballot b generated by an honest voter V , do not yet add this ballot to the state of the BB. Instead, instruct a malicious voter V' to copy the ciphertext and NIZKP contained in b and sign it under the name of the malicious voter to create a copy b' . Then add both b' and b to the BB (after setup, which has already finished when honest voters start creating their ballots, and before the `VotingClosed` message) such that b' appears before b . Vote verification will be successful for V , but their ballot will be dropped during ballot weeding.

Observe, however, that the ciphertext generated by V , which contains their intended vote, will still be counted towards the final result, just now in the name of a malicious voter. This is equivalent (in the sense of yielding the same vote count) to the case where b is counted for V while V' chooses to abstain, which is a correct vote distribution, and therefore does not break correctness of the election result. Formally, the intended vote by V is still included in the vector $\mathbf{vv}^{\text{real}}$, which does not make any statement about who submitted which ballot. Hence, the only way for $\mathbf{vv}^{\text{real}}$ to possibly deviate from a correct vote distribution $\mathbf{vv}^{\text{ideal}}$ is case (iii).

Case (iii): Given a (signed) ballot b generated by an honest voter V who did not choose to abstain, the adversary can choose to not include the unmodified b before the `VotingClosed` message on the BB. This, in turn, generally

results in this vote being dropped,¹² i.e., this case changes the distance to a correct result by at most 1 since it removes at most one honest vote from the distribution but cannot replace it with a different one.

Since we have that the distance of the actual votes $\mathbf{vv}^{\text{real}}$ to a correct vote distribution $\mathbf{vv}^{\text{ideal}}$ is larger than k and case (iii) is the only option to change the distance, it must be the case that at least $k + 1$ ballots from honest voters who did not choose to abstain were dropped in this manner. Observe that if an affected voter decides to perform vote verification, then this will cause a failure. In event χ_7 , vote verification succeeded for all honest voters who decided to perform it (cf. event χ_4), which implies that the affected (at least) $k + 1$ voters whose ballots were not included correctly did not decide to perform vote verification. As this decision is made independently from the rest of the system, this happens only with probability at most $(1 - p_{\text{verif}})^{k+1}$. Therefore we have $\Pr[\chi_7] \leq (1 - p_{\text{verif}})^{k+1} = \delta_k(p_{\text{verif}})$.

This concludes Step 1 by showing that $\Pr[\chi_{\text{ideal-BB}}]$ is $\delta_k(p_{\text{verif}})$ -bounded.

Step 2. Define the event $\chi_{\text{real-BB}}$ to be the set of runs of $\pi_{\text{real-BB}}(1^\eta)$ where $\neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$, i.e., analogous to $\chi_{\text{ideal-BB}}$ but taken over runs of the protocol with the real BB $\mathcal{P}_{\text{BB}}$. We want to show that the difference $|\Pr[\chi_{\text{real-BB}}] - \Pr[\chi_{\text{ideal-BB}}]|$ is upper bounded by a negligible function, which will conclude the proof. As already briefly mentioned, we will show this result by reducing to the UC indistinguishability of $\mathcal{P}_{\text{BB}}$ and $\mathcal{F}_{\text{BB}}$ running with a simulator $\mathcal{S}$ (cf. Figure 1). The main point that we need to establish in this reduction is that the trace property $\neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$ for our e-voting system can be decided by a ppt UC environment simulating the voting system while having only black-box access to either $\mathcal{P}_{\text{BB}}$ or $\mathcal{F}_{\text{BB}} \mid \mathcal{S}$.

We define the following distinguishing environment $\mathcal{E}$, which runs either with $\mathcal{P}_{\text{BB}}$ in the real world (controlling the inputs/outputs as well as the network traffic of $\mathcal{P}_{\text{BB}}$) or with $\mathcal{F}_{\text{BB}} \mid \mathcal{S}$ in the ideal world (controlling inputs/outputs of $\mathcal{F}_{\text{BB}}$ and the simulated network traffic of $\mathcal{S}$). The environment $\mathcal{E}$ simulates a run of $\pi_{\text{real-BB}} = (\mathcal{P}_{\text{ABOVE}} \mid \mathcal{A})$ including the network adversary $\mathcal{A}$ but excluding the bulletin board (cf. Figure 2). Instead, whenever the simulated protocol wants to send an input or receive an output to/from its bulletin board, $\mathcal{E}$ forwards/returns this to/from the UC protocol $\mathcal{P}_{\text{BB}}$ or $\mathcal{F}_{\text{BB}}$ with which it is running. Similarly, whenever the simulated $\mathcal{A}$ wants to send or receives a network message to/from (a party running) the bulletin board, $\mathcal{E}$ forwards this via the network connection to/from $\mathcal{P}_{\text{BB}}$ or $\mathcal{S}$. Hence, if $\mathcal{E}$ is running in the real world with $\mathcal{P}_{\text{BB}}$, it perfectly simulates a run of $\pi_{\text{real-BB}}$, but if it is running in the ideal world with $\mathcal{F}_{\text{BB}} \mid \mathcal{S}$, it perfectly simulates a run of $\pi_{\text{ideal-BB}}$ instead. Note that this simulation is possible in polynomial time.

After the simulation has stopped, $\mathcal{E}$ checks whether the property $\neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$ is met in the current run

¹²In principle, a malicious voter could choose to submit the same ciphertext and NIZKP but signed via their key. Then, the vote would still be counted.

and outputs 1 if and only if this is the case. Observe that $\mathcal{E}$ has access to all necessary information and can compute this property within polynomial time: $\mathcal{E}$ has full control over the judge J and, therefore, sees whether it has accepted the run. As for $\gamma_k^{\text{gen}}(\phi)$, the environment controls all parties in the voting protocol. It can, therefore, check whether any two honest parties disagree on the election result or the set of eligible voters. It can also easily verify the honesty assumptions (possibly by querying the bulletin board whether a subroutine client belonging to a party is currently corrupted. UC protocols offer an interface for this purpose). Since the environment controls all honest voters in particular, it further has access to the list $v_1, \dots, v_{n_{\text{voters}}^{\text{hon}}}$ of their intended votes. From this list and the election result¹³, the environment can check whether the result can be obtained from a vote distribution that has distance at most k from a correct distribution: the environment considers all possible vote distributions that yield the election result and all possible votes of malicious parties in a correct vote distribution and then checks whether there is a combination that meets the distance condition. While this is a brute force approach, since both the number of eligible voters $\text{id}_{\text{voters}}$ and the set of valid votes $\tilde{C}$ are fixed and finite, there are only finitely many options that are independent of the security parameter. Runtime is, therefore, (possibly large but) actually constant and, in particular, still ppt in the security parameter.

By its definition, $\mathcal{E}$ in a run with $\mathcal{P}_{\text{BB}}$ outputs 1 if and only if the simulated run belongs to $\chi_{\text{real-BB}}$. In a run with $\mathcal{F}_{\text{BB}} \mid \mathcal{S}$, $\mathcal{E}$ outputs 1 if and only if the simulated run belongs to $\chi_{\text{ideal-BB}}$. Since $\mathcal{E}$ is a ppt UC environment and $\mathcal{P}_{\text{BB}}$ is indistinguishable from $\mathcal{F}_{\text{BB}} \mid \mathcal{S}$ for any ppt UC environment by the assumption that $\mathcal{P}_{\text{BB}} \leq \mathcal{F}_{\text{BB}}$, we immediately obtain that $|\Pr[\chi_{\text{real-BB}}] - \Pr[\chi_{\text{ideal-BB}}]|$ is negligible. Combined with Step 1, which has established that $\chi_{\text{ideal-BB}}$ is $\delta_k(p_{\text{verif}})$ -bounded, this implies that $\Pr[\chi_{\text{real-BB}}]$ is $\delta_k(p_{\text{verif}})$ -bounded as well, thus concluding the proof. $\square$

APPENDIX F PROOF OF PRIVACY

Here, we provide the proof of Theorem 3. For this purpose, let $\mathcal{A}_0$ be an admissible ppt adversary on $\mathcal{P} = \text{ABOVE}$ with n_{voters} voters of which $n_{\text{voters}}^{\text{hon}}$ are non-abstaining honest voters (that is, they vote according to the distribution μ_{vote} on C), n_{talliers} talliers of which (at least) one is honest, and let v_{obs} denote the voter under observation.

Through several steps we will move from $(\mathcal{P} \mid \mathcal{A})$ to $(\mathcal{P}_{\text{ideal}} \mid \mathcal{A}_{\text{ideal}})$, where $\mathcal{P}_{\text{ideal}}$ denotes the ideal voting protocol with n_{voters} voters of which (at least) $n_{\text{voters}}^{\text{hon}} - k$ are

¹³Referring to “the” election result is justified at this point: If there is no election result, then the judge will not be able to compute a result and not accept; in this case, the environment can already conclude that the run is not in $\neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$. If the election result exists but is not uniquely defined because different honest parties computed different results, then the environment knows that the run is in $\neg\gamma_k^{\text{gen}}(\phi)$ and can decide $\neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$ based on the output of the judge. Hence, if an environment reaches this point and still has not decided yet whether the current run is in $\neg\gamma_k^{\text{gen}}(\phi) \wedge (J : \text{accept})$, then there exists one uniquely defined election result.

non-abstaining honest voters (cf. Figure 3) and $\mathcal{A}_{\text{ideal}}$ is a ppt adversary for $\mathcal{P}_{\text{ideal}}$. We will prove that each step only negligibly affects the adversary’s winning probability in the corresponding privacy game, thus showing that

$|\Pr[\mathbb{E}_0^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}}) \mapsto 1] - \Pr[\mathbb{E}_0^{\text{Priv}}(\mathcal{A}_{\text{ideal}}, \mathcal{P}_{\text{ideal}}, \eta, v_{\text{obs}}) \mapsto 1]|$ is negligible in η , i.e., that ABOVE achieves (ideal) privacy. We denote $\mathbb{E}_0^{\text{Priv}} := \mathbb{E}_0^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, v_{\text{obs}})$. Similarly, if in any step i the parameters of a privacy game $\mathbb{E}_i^{\text{Priv}}(\mathcal{A}_i, \mathcal{P}_i, \eta, v_{\text{obs}})$ are clear from the context, we will sometimes write $\mathbb{E}_i^{\text{Priv}}(\mathcal{A}_i)$ or $\mathbb{E}_i^{\text{Priv}}$ for short.

Step $\mathbb{E}_0^{\text{Priv}} \rightarrow \mathbb{E}_1^{\text{Priv}}$: Unforged Signatures. We define $\mathbb{E}_1^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_0^{\text{Priv}}$ in the following way: $\mathcal{P}$ halts if at any point a signed message x with signature σ occurs in any honest party’s view of the BB such that $\text{Verify}(\text{pk}, x, \sigma) = \text{true}$ (where pk denotes an honest party’s public signing key) but x was not signed using a secret signing key sk such that (pk, sk) was obtained by invoking the key generation algorithm of the digital signature scheme. That is, whenever a forged signature occurs, the process halts. Since by assumption (P3) the used signature scheme $\mathcal{S}$ is EUF-CMA-secure (cf. Section C-A), such forgery can only occur with probability negligible in η . Hence, the probability that $\mathcal{P}$ halts due to this change is negligible. In particular, for any admissible ppt adversary $\mathcal{A}_1$ for $\mathbb{E}_1^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_0^{\text{Priv}}(\mathcal{A}_0) \mapsto 1] - \Pr[\mathbb{E}_1^{\text{Priv}}(\mathcal{A}_1) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_1^{\text{Priv}} \rightarrow \mathbb{E}_2^{\text{Priv}}$: Ciphertext Collisions. We define $\mathbb{E}_2^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_1^{\text{Priv}}$ in the following way: $\mathcal{P}$ halts if at any point two or more honest (non-abstaining) voters compute the same ciphertext vector ct . As argued in the proof of E2E verifiability (cf. Footnote 11 in Appendix E), the probability that $\mathcal{P}$ halts due to this change is negligible since, by assumption (P2), we consider a non-trivial ElGamal group $\mathbb{G}$ in which the DDH problem is hard. In particular, for any adversary $\mathcal{A}_2$ for $\mathbb{E}_2^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_1^{\text{Priv}}(\mathcal{A}_1) \mapsto 1] - \Pr[\mathbb{E}_2^{\text{Priv}}(\mathcal{A}_2) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_2^{\text{Priv}} \rightarrow \mathbb{E}_3^{\text{Priv}}$: k -risk avoiding adversaries. We define $\mathbb{E}_3^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_2^{\text{Priv}}$ in the following way: $\mathcal{P}$ halt if the BB provides inconsistent views to two (or more) honest parties and/or if an output returned by the BB to an honest party contains a successful setup phase and a valid signed VotingClosed message but in that output and before the first VotingClosed message there are less than $n_{\text{voters}}^{\text{hon}} - k$ signed ballots submitted by honest voters.

Since we assume $\mathcal{A}$ to be admissible, it is, in particular, k -risk avoiding (cf. assumption (P1)). Hence, the probability that $\mathcal{P}$ halts due to this change is negligible and, in particular, for any admissible ppt adversary $\mathcal{A}_3$ for $\mathbb{E}_3^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_2^{\text{Priv}}(\mathcal{A}_2) \mapsto 1] - \Pr[\mathbb{E}_3^{\text{Priv}}(\mathcal{A}_3) \mapsto 1]|$ is negligible in η . We note that due to this change, we can, for the next steps, conclude the following (unless $\mathcal{P}$ halts): If a set of vectors of partial decryptions that yield an election result is contained in an honest party’s view of the BB, then there is (at most) one unique election result in all honest parties’ views of the BB and, similarly, the set of eligible voters and talliers (alongside their public signing keys) is well-defined - otherwise, the BB

would have provided inconsistent views to two or more honest parties. We can also conclude that (if an election result is contained in an honest party's view of the BB) at least $n_{\text{voters}}^{\text{hon}} - k$ signed ballots submitted by honest voters are contained in that party's view after the setup has finished and before the first `VotingClosed` message (cf. Step $\mathbb{E}_0^{\text{Priv}} \rightarrow \mathbb{E}_1^{\text{Priv}}$).¹⁴

Moreover, let T_{hon} be an arbitrary but fixed honest tallier that exists by assumption (P1). If an election result is contained in any honest party's view of the BB, then T_{hon} has a view of the BB that contains all setup messages, the set $\mathcal{B}$ of all (signed) ballots that are considered for computing the election result¹⁵, and all corresponding NIZKPs (which includes π^{Enc} for all voters, and π^{DecShare} , and $\pi^{\text{KeyShareGen}}$ for all talliers). Otherwise T_{hon} would not compute a partial decryption dec_{hon} and, hence, no election result could be computed by the talliers specified in the signed setup message from Auth (which is not forged by Step $\mathbb{E}_0^{\text{Priv}} \rightarrow \mathbb{E}_1^{\text{Priv}}$). Moreover, for the same reason, if an election result is contained in an honest party's view of the BB, then all NIZKPs contained in T_{hon} 's view of the BB must be valid.

Step $\mathbb{E}_3^{\text{Priv}} \rightarrow \mathbb{E}_4^{\text{Priv}}$: Simulate T_{hon} 's NIZKPs. We define $\mathbb{E}_4^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_3^{\text{Priv}}$ in the following way: For all NIZKPs computed by T_{hon} , we instead let Sch run the zero knowledge simulator (cf. Section C-B) to obtain proofs π^{DecShare} and $\pi^{\text{KeyShareGen}}$ and forward them to T_{hon} . Instead of computing NIZKPs on its own, T_{hon} uses the simulated proofs. By the zero-knowledge property of the two NIZKPs, no ppt adversary can distinguish between real and simulated proofs with non-negligible probability. Thus, for any admissible ppt adversary $\mathcal{A}_4$ for $\mathbb{E}_4^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_3^{\text{Priv}}(\mathcal{A}_3) \mapsto 1] - \Pr[\mathbb{E}_4^{\text{Priv}}(\mathcal{A}_4) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_4^{\text{Priv}} \rightarrow \mathbb{E}_5^{\text{Priv}}$: Decryption key shares of dishonest talliers. We define $\mathbb{E}_5^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_4^{\text{Priv}}$ in the following way: Using the extractor algorithm for the NIZKPs for proving knowledge and correctness of a private decryption key share, extract the secret key shares of all dishonest talliers from all valid NIZKPs $\pi^{\text{KeyShareGen}}$ contained in the BB view seen by T_{hon} and forward them to T_{hon} . If the extraction fails, the process halts. We note that by the simulation sound extractability of the NIZKPs (cf. Section C-B), the extraction fails only with negligible probability (for more details, see the reasoning for the event χ_5 having negligible probability in Appendix E). Moreover, if the process does not halt, nothing changes from an adversary's point of view, since the extraction and forwarding of the secret key shares happens internally. Thus, for any admissible ppt adversary $\mathcal{A}_5$ for $\mathbb{E}_5^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_4^{\text{Priv}}(\mathcal{A}_4) \mapsto 1] - \Pr[\mathbb{E}_5^{\text{Priv}}(\mathcal{A}_5) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_5^{\text{Priv}} \rightarrow \mathbb{E}_6^{\text{Priv}}$: Honest voters use simulated NIZKPs. We define $\mathbb{E}_6^{\text{Priv}}$ as the game that we obtain by modifying

$\mathbb{E}_5^{\text{Priv}}$ in the following way: For all NIZKPs that would be computed by honest voters, the scheduler Sch runs the zero knowledge simulator (cf. Section C-B) to obtain proofs π^{Enc} and forwards them to the corresponding voters. Instead of computing NIZKPs on their own, the honest voters include the simulated proofs in their ballots. By the zero-knowledge property of the NIZKP for proving knowledge of a (valid) plaintext vote, no adversary can distinguish between real and simulated proofs with non-negligible probability. Thus, for any admissible ppt adversary $\mathcal{A}_6$ for $\mathbb{E}_6^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_5^{\text{Priv}}(\mathcal{A}_5) \mapsto 1] - \Pr[\mathbb{E}_6^{\text{Priv}}(\mathcal{A}_6) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_6^{\text{Priv}} \rightarrow \mathbb{E}_7^{\text{Priv}}$: Extract votes of dishonest voters. Consider the set $\mathcal{B}$ consisting of all signed ballots taken into account for computing the election result (if an election result is computed), i.e., those ballots contain a ciphertext vector ct that is included in the overall aggregated ciphertext vector ct_{aggr} . Note that this is indeed a set and it does not contain any duplicate ciphertexts. This is due to the ballot weeding process described in Section III.

We partition $\mathcal{B}$ into two subsets: (i) $\mathcal{B}_1 \subseteq \mathcal{B}$ consisting of all ballots in $\mathcal{B}$ that contain a ciphertext vector ct that was generated by an honest voter. (ii) $\mathcal{B}_2 \subseteq \mathcal{B}$ consisting of all ballots in $\mathcal{B}$ that contain a ciphertext vector ct that was not generated by an honest voter.

We define $\mathbb{E}_7^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_6^{\text{Priv}}$ in the following way: For each ballot occurring in T_{hon} 's view of the BB that belongs to $\mathcal{B}_2$, use the extractor algorithm for the NIZKPs for proving knowledge of a (valid) plaintext vote to extract the vote $\mathbf{v}$ from π^{Enc} and forward $\mathbf{v}$ to T_{hon} . If the extraction fails, the process halts. As in Step $\mathbb{E}_4^{\text{Priv}} \rightarrow \mathbb{E}_5^{\text{Priv}}$, the simulation sound extractability guarantees that the extraction fails only with negligible probability. Moreover, if the process does not halt, nothing changes from an adversary's point of view since the extraction and forwarding of the secret key shares happens internally. Thus, for any admissible ppt adversary $\mathcal{A}_7$ for $\mathbb{E}_7^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_6^{\text{Priv}}(\mathcal{A}_6) \mapsto 1] - \Pr[\mathbb{E}_7^{\text{Priv}}(\mathcal{A}_7) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_7^{\text{Priv}} \rightarrow \mathbb{E}_8^{\text{Priv}}$: Dishonest talliers' partial decryptions. We define $\mathbb{E}_8^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_7^{\text{Priv}}$ in the following way: If T_{hon} 's starts tallying and obtains an aggregated ciphertext (and hence, in particular, sees a successful setup phase and a `VotingClosed` message), let T_{hon} use the extracted dishonest talliers' secret key shares to decrypt the aggregated ciphertext ct_{aggr} to obtain a vector ds_i of partial decryptions for each dishonest tallier T_i . This does not change anything from an adversary's point of view. Thus, for any admissible ppt adversary $\mathcal{A}_8$ for $\mathbb{E}_8^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_7^{\text{Priv}}(\mathcal{A}_7) \mapsto 1] - \Pr[\mathbb{E}_8^{\text{Priv}}(\mathcal{A}_8) \mapsto 1]|$ is negligible in η .

Step $\mathbb{E}_8^{\text{Priv}} \rightarrow \mathbb{E}_9^{\text{Priv}}$: Modify election result computation. The goal of this step is to change T_{hon} such that they no longer use their secret key share $\text{sk}_{\text{hon}}^{\text{E}}$. We have already made T_{hon} 's NIZKP $\pi^{\text{KeyShareGen}}$ independent of $\text{sk}_{\text{hon}}^{\text{E}}$ (Step $\mathbb{E}_3^{\text{Priv}} \rightarrow \mathbb{E}_4^{\text{Priv}}$), but in $\mathbb{E}_8^{\text{Priv}}$, T_{hon} still needs $\text{sk}_{\text{hon}}^{\text{E}}$ for computing their vector ds_{hon} of partial decryption results. For this purpose, we describe a way for T_{hon} to compute ds_{hon} using the intended

¹⁴Note that k -risk avoiding adversaries only guarantee that these votes appear before the `VotingClosed` message. However, we can reason as in the discussion of (i) for event χ_7 in Appendix E to see that the $n_{\text{voters}}^{\text{hon}} - k$ signed ballots submitted by honest voters do indeed appear after the setup finished. Otherwise the process would have halted due to a forged signature.

¹⁵We also let T_{hon} forward $\mathcal{B}$ to the scheduler Sch for a later step.

votes of honest voters, the extracted votes of dishonest voters, and the other talliers' vectors of partial decryptions $\mathbf{ds}_i$.

Concretely, we define $\mathbb{E}_9^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_8^{\text{Priv}}$ in the following way: First, we let each honest voter V_i forward their intended (plain) vote as well as the corresponding ciphertext vector $\mathbf{ct}_i$ internally to the scheduler Sch. From these ciphertexts and $\mathfrak{B}$, Sch can compute the partition $\mathfrak{B} = \mathfrak{B}_1 \cup \mathfrak{B}_2$ introduced in step $\mathbb{E}_6^{\text{Priv}} \rightarrow \mathbb{E}_7^{\text{Priv}}$. The scheduler then computes the “target election result” $\widetilde{\text{res}} = (\widetilde{\text{res}}[j])_{j=1}^{n_{\text{cand}}}$ from $\mathfrak{B}$ as follows: For each (signed) ballot $b \in \mathfrak{B}_1$, count the intended plain vote from the honest voter who sent the ciphertext vector included in b (it does not matter who signed b) and for each ballot in $\mathfrak{B}_2$, count the corresponding vote extracted as described in $\mathbb{E}_6^{\text{Priv}} \rightarrow \mathbb{E}_7^{\text{Priv}}$. Then, Sch forwards $\widetilde{\text{res}}$ to T_{hon} .

Now, if T_{hon} would compute a vector $\mathbf{ds}_{\text{hon}}$ of partial decryptions, they instead compute a vector $\widetilde{\mathbf{ds}}_{\text{hon}}$ of “fake” partial decryptions as follows: Let $\mathbf{ct}_{\text{aggr}}$ be the aggregation of the ciphertext vectors taken into account for computing the election result with entries $\mathbf{ct}_{\text{aggr}}[j] = (h_{1,j}, h_{2,j})$ for candidate/option j . Then, the j -th entry $\widetilde{\mathbf{ds}}_{\text{hon}}[j]$ of $\widetilde{\mathbf{ds}}_{\text{hon}}$ is computed as

$$\widetilde{\mathbf{ds}}_{\text{hon}}[j] = h_{2,j} \cdot \left(\prod_{i \in \{1, \dots, n_{\text{talliers}}\}, T_i \neq T_{\text{hon}}} \mathbf{ds}_i[j] \right)^{-1} \cdot g^{-\widetilde{\text{res}}[j]},$$

where $\mathbf{ds}_i[j]$ denotes the j -th entry of T_i 's vector of partial decryptions. Note that $\mathbf{ds}_i$ was either extracted in the previous step or can be computed by another honest tallier (and forwarded to T_{hon}). Instead of its “real” vector $\mathbf{ds}_{\text{hon}}$ of partial decryption results, T_{hon} then publishes $\widetilde{\mathbf{ds}}_{\text{hon}}$. We note that, by construction, it holds true that $\mathbf{ds}_{\text{hon}} = \widetilde{\mathbf{ds}}_{\text{hon}}$. Hence, this adaptation of T_{hon} does not change anything from an adversary's point of view. Thus, for any admissible ppt adversary $\mathcal{A}_9$ for $\mathbb{E}_9^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_8^{\text{Priv}}(\mathcal{A}_8) \mapsto 1] - \Pr[\mathbb{E}_9^{\text{Priv}}(\mathcal{A}_9) \mapsto 1]|$ is negligible in η .

Observe that now, indeed, all computations in $\mathbb{E}_9^{\text{Priv}}$ are independent of T_{hon} 's secret key share $\text{sk}_{\text{hon}}^\mathcal{E}$.

Step $\mathbb{E}_9^{\text{Priv}} \rightarrow \mathbb{E}_{10}^{\text{Priv}}$: Honest voters use dummy ciphertexts. We define $\mathbb{E}_{10}^{\text{Priv}}$ as the game that we obtain by modifying $\mathbb{E}_9^{\text{Priv}}$ in the following way: All honest voters compute a ciphertext vector $\widetilde{\mathbf{ct}} \in \mathbb{G}^{n_{\text{cand}}}$ that contains encryptions of 0 (or any other dummy value, but we will use 0 for ease of notation) in each entry and then include $\widetilde{\mathbf{ct}}$ in their ballot instead of the ciphertext vector $\mathbf{ct}$ consisting of encryptions of their intended vote.

In order to see that this change only negligibly affects an adversary $\mathcal{A}$'s winning probability, we construct an adversary $\mathcal{A}_{\text{EG}}$ that wins the IND-CPA game against the (standard, i.e., non-threshold) ElGamal challenger over the group $\mathbb{G}$ if the difference $|\Pr[\mathbb{E}_9^{\text{Priv}}(\mathcal{A}) \mapsto 1] - \Pr[\mathbb{E}_{10}^{\text{Priv}}(\mathcal{A}) \mapsto 1]|$ is non-negligible. Since ElGamal is IND-CPA-secure under the DDH-assumption (cf. Section C-C), this shows that, by assumption (P2), $\mathbb{E}_9^{\text{Priv}}$ and $\mathbb{E}_{10}^{\text{Priv}}$ are computationally indistinguishable.

For simplicity, we first argue that $\mathbb{E}_9^{\text{Priv}}$ and $\widetilde{\mathbb{E}}_{10}^{\text{Priv}}$ are computationally indistinguishable, where $\widetilde{\mathbb{E}}_{10}^{\text{Priv}}$ is defined as the game where only one voter V_i changes only the first entry of its ciphertext vector $\mathbf{ct}_i$ (containing V_i 's intended vote) to an encryption of 0. That is, $\widetilde{\mathbf{ct}}'_i[1]$ is an encryption of 0, while $\widetilde{\mathbf{ct}}'_i[j] = \mathbf{ct}_i[j]$ for $j = 2, \dots, n_{\text{cand}}$ and $\widetilde{\mathbf{ct}}_k = \mathbf{ct}_k$ for $k \neq i$. Applying this result iteratively (entry-by-entry and voter-by-voter) yields the overall indistinguishability result between $\mathbb{E}_9^{\text{Priv}}$ and $\mathbb{E}_{10}^{\text{Priv}}$. We base our reduction on the one for Belenios ElGamal presented in [3] while adapting it to our full-threshold setting. For this purpose, we construct $\mathcal{A}_{\text{EG}}$ as follows:

In the IND-CPA game, $\mathcal{A}_{\text{EG}}$ first receives ElGamal parameters $(\mathbb{G}, p, g)$ and an ElGamal public key $K^* \in \mathbb{G}$ from the ElGamal challenger. We then let $\mathcal{A}_{\text{EG}}$ play the role of the challenger $\mathcal{CH}$ in $\mathbb{E}_9^{\text{Priv}}$ resp. $\widetilde{\mathbb{E}}_{10}^{\text{Priv}}$ with $\mathcal{A}$. $\mathcal{CH}$ lets Auth set up the ElGamal parameters as $(\mathbb{G}, p, g)$ and lets T_{hon} use K^* as its public key share $\text{pk}_{\text{hon}}^\mathcal{E}$. Since K^* is uniformly distributed in $\mathbb{G}$ and all computations in $\mathbb{E}_9^{\text{Priv}}$ are independent of T_{hon} 's secret key share $\text{sk}_{\text{hon}}^\mathcal{E}$, this behavior by T_{hon} is indistinguishable from its behavior in $\mathbb{E}_9^{\text{Priv}}$ resp. $\widetilde{\mathbb{E}}_{10}^{\text{Priv}}$. All other honest talliers T_i create their public key shares $\text{pk}_i^\mathcal{E} = g^{\text{sk}_i^\mathcal{E}}$ as usual. $\mathcal{A}$ can have all dishonest talliers T_i publish their (potentially dishonestly generated) public key shares $h_i (= g^{\text{sk}_i^{\text{dishon}}})$ for some $\text{sk}_i^{\text{dishon}} \in \mathbb{Z}_p$ unknown to the honest parties) alongside their NIZKP $\pi_i^{\text{KeyShareGen}}$. If the setup is successful, this yields an overall public encryption key

$$\text{pk}^\mathcal{E} = K^* \cdot \prod_{\substack{T_i \text{ honest,} \\ T_i \neq T_{\text{hon}}}} \text{pk}_i^\mathcal{E} \cdot \prod_{T_i \text{ dishonest}} h_i$$

of the election. In runs where the election's setup finishes successfully, we know that the $\pi_i^{\text{KeyShareGen}}$ are valid. Hence, in such runs, $\mathcal{A}_{\text{EG}}$ can extract the secret key shares $\text{sk}_i^{\text{dishon}}$ for all dishonest talliers.

When V_i wants to encrypt their vote, $\mathcal{A}_{\text{EG}}$ chooses $m_0 = 0$ (the *dummy message*) and $m_1 = v_i[1]$ (the first entry of V_i 's intended vote) from $\mathbb{Z}_p$ and sends them to the ElGamal challenger, who flips a bit $b \leftarrow_{\$} \{0, 1\}$ and returns an (Exponential) ElGamal ciphertext $\mathbf{ct}_1^i = (c_0, c_1) = (g^r, g^{m_b} \cdot (K^*)^r)$ for some (unknown) $r \leftarrow_{\$} \mathbb{Z}_p$ to $\mathcal{A}_{\text{EG}}$. $\mathcal{A}_{\text{EG}}$ then computes

$$\widetilde{c}_1 := c_1 \cdot c_0^{\left(\sum_{\substack{T_i \text{ honest,} \\ T_i \neq T_{\text{hon}}}} \text{sk}_i^\mathcal{E} + \sum_{T_i \text{ dishonest}} \text{sk}_i^{\text{dishon}} \right)},$$

sets $\mathbf{ct}_1^i := (c_0, \widetilde{c}_1)$ and forwards $\mathbf{ct}_1^i$ to V_i who produces their ciphertext vector $\mathbf{ct}_i$ as usual but replaces the first entry with $\mathbf{ct}_1^i$ to obtain a ciphertext vector $\widetilde{\mathbf{ct}}_i$. Since V_i is honest, it uses a simulated NIZKP π_i^{Enc} and, hence, all further steps can still be computed without knowing the plaintext corresponding to $\widetilde{\mathbf{ct}}_i$. Furthermore, note that $\mathbf{ct}_1^i$ is a valid encryption of m_b under the public encryption key $\text{pk}^\mathcal{E}$ of the election. Indeed,

we have that

$$\begin{aligned}
\tilde{c}_1 &= c_1 \cdot c_0 \left(\sum_{\substack{T_i \text{ honest,} \\ T_i \neq T_{\text{hon}}}} \text{sk}_i^\varepsilon + \sum_{\substack{T_i \text{ dis-} \\ \text{honest}}} \text{sk}_i^{\text{dishon}} \right) \\
&= g^{m_b} \cdot (K^*)^r \cdot \left(\prod_{\substack{T_i \text{ honest,} \\ T_i \neq T_{\text{hon}}}} g^{\text{sk}_i^\varepsilon} \cdot \prod_{\substack{T_i \text{ dis-} \\ \text{honest}}} g^{\text{sk}_i^{\text{dishon}}} \right)^r \\
&= g^{m_b} \cdot \left(K^* \cdot \prod_{\substack{T_i \text{ honest,} \\ T_i \neq T_{\text{hon}}}} g^{\text{sk}_i^\varepsilon} \cdot \prod_{\substack{T_i \text{ dis-} \\ \text{honest}}} g^{\text{sk}_i^{\text{dishon}}} \right)^r = g^{m_b} \cdot (\text{pk}^\varepsilon)^r
\end{aligned}$$

i.e., $\text{ct}_1^i = (g^r, g^{m_b} \cdot (\text{pk}^\varepsilon)^r)$. Thus, from $\mathcal{A}$'s point of view, we are exactly in $\mathbb{E}_9^{\text{Priv}}$ or $\mathbb{E}_{10}^{\text{Priv}}$. After the protocol finishes, $\mathcal{A}$ returns its guess b' to $\mathcal{A}_{\text{EG}}$, who forwards b' to the ElGamal challenger. Now, if the difference $|\Pr[\mathbb{E}_9^{\text{Priv}}(\mathcal{A}) \mapsto 1] - \Pr[\mathbb{E}_{10}^{\text{Priv}}(\mathcal{A}) \mapsto 1]|$ is non-negligible, we in turn get that

$$\begin{aligned}
&|\Pr[(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\eta); (m_0, m_1) \leftarrow \mathcal{A}_{\text{EG}}(1^\eta, \text{pk}); b = 1; \\
&\quad \text{ct}_b \leftarrow \text{Enc}^{\text{pk}}(x_b); b' \leftarrow \mathcal{A}_{\text{EG}}(1^\eta, \text{pk}, \text{ct}_b); b = b'] \\
&- \Pr[(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\eta); (m_0, m_1) \leftarrow \mathcal{A}_{\text{EG}}(1^\eta, \text{pk}); b = 0; \\
&\quad \text{ct}_b \leftarrow \text{Enc}^{\text{pk}}(x_b); b' \leftarrow \mathcal{A}_{\text{EG}}(1^\eta, \text{pk}, \text{ct}_b); b \neq b']|
\end{aligned}$$

is non-negligible, showing that $\mathcal{A}_{\text{EG}}$ wins the IND-CPA game with non-negligible probability.

Applying this argument iteratively to each voter and each vector entry (i.e., a constant number of times), we get that $\mathbb{E}_9^{\text{Priv}}$ and $\mathbb{E}_{10}^{\text{Priv}}$ are computationally indistinguishable if ElGamal is IND-CPA secure, i.e., if the DDH assumption holds for $\mathbb{G}$. As this holds by (P2), we find that for any admissible ppt adversary $\mathcal{A}_{10}$ for $\mathbb{E}_{10}^{\text{Priv}}$, we get that $|\Pr[\mathbb{E}_9^{\text{Priv}}(\mathcal{A}_9) \mapsto 1] - \Pr[\mathbb{E}_{10}^{\text{Priv}}(\mathcal{A}_{10}) \mapsto 1]|$ is negligible in η .

Final Step: Constructing $\mathcal{A}_{\text{ideal}}$. The previous steps have shown that for any admissible adversary $\mathcal{A}$ against ABOVE, there is an adversary $\mathcal{A}_{10}$ such that the difference $|\Pr[\mathbb{E}^{\text{Priv}}(\mathcal{A}, \mathcal{P}, \eta, \mathbf{v}_{\text{obs}}) \mapsto 1] - \Pr[\mathbb{E}_{10}^{\text{Priv}}(\mathcal{A}_{10}) \mapsto 1]|$ is negligible in η . We now show that, using any such (admissible) adversary $\mathcal{A}_{10}$, we can construct an adversary $\mathcal{A}_{\text{ideal}}$ against $\mathcal{P}_{\text{ideal}} := \mathcal{P}_{\text{ideal}}(n_{\text{voters}}, n_{\text{voters}}^{\text{hon}} - k, C, \mu_{\text{vote}})$ such that the difference $|\Pr[\mathbb{E}_{10}^{\text{Priv}}(\mathcal{A}_{10}) \mapsto 1] - \Pr[\mathbb{E}_{11}^{\text{Priv}}(\mathcal{A}_{\text{ideal}}) \mapsto 1]|$ is negligible in η (where $\mathbb{E}_{11}^{\text{Priv}} := \mathbb{E}^{\text{Priv}}(\mathcal{A}_{\text{ideal}}, \mathcal{P}_{\text{ideal}}, \eta, \mathbf{v}_{\text{obs}})$). Combining these two results will complete the proof of Section VI-C by the triangle inequality.

The main idea is as follows: Observe that in $\mathbb{E}_{10}^{\text{Priv}}$ the adversary and protocol are almost independent of the intended votes of the honest voters including $\mathbf{v}_{\text{obs}}$. The only step that still depends on this information is the computation of the partial decryptions of T_{hon} , which only requires the election result, not individual intended votes. We can therefore define $\mathcal{A}_{\text{ideal}}$ to simulate both adversary and protocol from $\mathbb{E}_{10}^{\text{Priv}}$ while

using $\mathcal{P}_{\text{ideal}}$ only to learn the election result. To show that this change is indistinguishable we only have to argue that the election result in $\mathbb{E}_{10}^{\text{Priv}}$ contains (at most) one copy of the vote of $\mathbf{v}_{\text{obs}}$ and at least $n_{\text{voters}}^{\text{hon}} - k - 1$ independent votes from (non-abstaining) honest voters.

More formally, let $\mathcal{A}_{\text{ideal}}$ simulate the protocol and adversary $\mathcal{A}_{10}$ for $\mathbb{E}_{10}^{\text{Priv}}$ (except for the honest voters' vote generation) until T_{hon} needs the target election result. For now we discuss the non-trivial case that $\mathfrak{B}$ in $\mathbb{E}_{10}^{\text{Priv}}$ contains a ciphertext generated by $\mathbf{v}_{\text{obs}}$ (we will discuss the edge case where $\mathfrak{B}$ contains no such ballot in the end of this proof.) Observe that, since we admit only k -risk avoiding adversaries (cf. $\mathbb{E}_2^{\text{Priv}} \rightarrow \mathbb{E}_3^{\text{Priv}}$) and ciphertexts of honest voters do not collide (cf. $\mathbb{E}_1^{\text{Priv}} \rightarrow \mathbb{E}_2^{\text{Priv}}$) such that they are not removed due to ballot weeding, there are at least $n_{\text{voters}}^{\text{hon}} - k$ votes in $\mathfrak{B}_1$. Due to ballot weeding, we also have that each ciphertext generated by an honest voter (in particular, a ciphertext containing the vote of $\mathbf{v}_{\text{obs}}$) is contained at most once in $\mathfrak{B}_1$. Instead of computing the election result as described in the step $\mathbb{E}_8^{\text{Priv}} \rightarrow \mathbb{E}_9^{\text{Priv}}$, we can thus define $\mathcal{A}_{\text{ideal}}$ to send (init, honest) to $\mathcal{P}_{\text{ideal}}$. In turn, $\mathcal{P}_{\text{ideal}}$ samples $n_{\text{voters}}^{\text{hon}} - k - 1$ honest votes from μ_{vote} , sets $\mathbf{v}_{\text{obs}}$'s vote to $\mathbf{v}_b$ and stores all of these votes. If $|\mathfrak{B}_1| = n_{\text{voters}}^{\text{hon}} - k + \lambda$ for some $\lambda > 0$ (i.e., there are more than $n_{\text{voters}}^{\text{hon}} - k$ ciphertexts generated by honest voters in $\mathfrak{B}$), $\mathcal{A}_{\text{ideal}}$ additionally samples λ votes $\mathbf{v}_i$ from μ_{vote} and sends (setChoice, $i, \mathbf{v}_i$) to $\mathcal{P}_{\text{ideal}}$ for $i = n_{\text{voters}}^{\text{hon}} - k + 1, \dots, n_{\text{voters}}^{\text{hon}} - k + \lambda$. This ensures that all honestly generated votes that are included in the election result in $\mathbb{E}_{10}^{\text{Priv}}$ are also included in the election result in $\mathcal{P}_{\text{ideal}}$. Finally, for any ciphertext in $\mathfrak{B}_2$ from a dishonest voter V_i , let $\mathbf{v}_i$ be the corresponding extracted vote (cf. Step $\mathbb{E}_6^{\text{Priv}} \rightarrow \mathbb{E}_7^{\text{Priv}}$) and let $\mathcal{A}_{\text{ideal}}$ send (setChoice, $i, \mathbf{v}_i$) to Sch for all such voters V_i .

After all votes have been registered correctly in $\mathcal{P}_{\text{ideal}}$, $\mathcal{A}_{\text{ideal}}$ sends compute and obtains an election result res from the challenger in $\mathbb{E}_{11}^{\text{Priv}}$. We note that, by construction and the above discussion, we have that res and $\widetilde{\text{res}}$ are distributed in the same way, where $\widetilde{\text{res}}$ denotes the "target election result" in $\mathbb{E}_{10}^{\text{Priv}}$. $\mathcal{A}_{\text{ideal}}$ therefore forwards res to T_{hon} and the rest of $\mathbb{E}_{10}^{\text{Priv}}$ is executed as normal. When $\mathcal{A}_{10}$ returns its guess b' , $\mathcal{A}_{\text{ideal}}$ forwards b' to the challenger in $\mathbb{E}_{11}^{\text{Priv}}$.

Since both, $\mathbb{E}_{10}^{\text{Priv}}$ and $\mathbb{E}_{11}^{\text{Priv}}$ used the same vote $\mathbf{v}_b$ for $\mathbf{v}_{\text{obs}}$, $\mathbb{E}_{10}^{\text{Priv}}$ returns 1 if and only if $\mathbb{E}_{11}^{\text{Priv}}$ returns 1. This shows that $|\Pr[\mathbb{E}_{10}^{\text{Priv}}(\mathcal{A}_{10}) \mapsto 1] - \Pr[\mathbb{E}_{11}^{\text{Priv}}(\mathcal{A}_{\text{ideal}}) \mapsto 1]| = 0$, which is of course negligible in η .

Above, we discussed the non-trivial case where the ciphertext vector ct generated by $\mathbf{v}_{\text{obs}}$ is included in $\mathfrak{B}$. Formally, it can also happen that in a run r the set $\mathfrak{B}$ does not contain ct because the adversary decided to drop it. Observe that in this case the system from $\mathbb{E}_{10}^{\text{Priv}}$ never makes use of $\mathbf{v}_b$: this value is only used when T_{hon} tallies and only if ct is part of the set of ciphertext vectors that are aggregated. Since $\mathbf{v}_b$ is the only information that is exclusive to $\mathcal{P}_{\text{ideal}}$, $\mathcal{A}_{\text{ideal}}$ can trivially and perfectly simulate such a run r without even using $\mathcal{P}_{\text{ideal}}$. Formally, $\mathcal{A}_{\text{ideal}}$ runs until T_{hon} starts tallying. It then checks whether $\text{ct} \in \mathfrak{B}$ and, if so, proceeds as above. If not, however, it ignores $\mathcal{P}_{\text{ideal}}$ and instead internally samples votes

$\mathbf{v}_i$ according to μ_{vote} for each ciphertext vector contained in $\mathfrak{B}_1$. The ideal adversary $\mathcal{A}_{\text{ideal}}$ then uses those votes $\mathbf{v}_i$ to complete the simulation of T_{hon} in the same way as $\mathbb{E}_{10}^{\text{Priv}}$ proceeds. Observe that the simulation of such runs is indeed perfect: The only technical difference is the point in time when the votes $\mathbf{v}_i$ are sampled. This, however, does not affect the behavior of the run since they are never used before T_{hon} starts tallying. $\square$